

PROJECTOPGAVE

SYSTEEMPROGRAMMEREN

prof. dr. ir. Filip De Turck
prof. dr. Bruno Volckaert

sysprog@lists.ugent.be

2de Bachelor Informatica
Academiejaar 2022-2023

INHOUDSOPGAVE

1	Inleiding	4
1.1	Algemene projectinformatie	4
2	Deel 1: Opgave	5
2.1	Levels omzetten	5
2.1.1	Implementatie tip: Stappen tijdelijk overslaan	5
2.1.2	Demo level	5
2.2	Spelregels spg implementeren a.d.h.v entity system framework (in C)	5
2.2.1	Spelregels	6
2.2.2	Entity System Framework	7
2.3	Level Editor	7
2.4	Command line arguments	8
2.5	Testen	9
3	Deel 2: Opgave	10
3.1	Aangepast Spel: spg_part2	10
3.2	Assembler	10
3.3	Emulator	13
3.4	CPU	14
3.5	Binaire CPU Instructies	15
3.5.1	Binair Formaat CPU Instructies	16
3.5.2	Hulpklassen voor het schrijven en lezen van bits	18
3.6	Beschrijving CPU Instructies	18
3.6.1	NOP	18
3.6.2	MOV	19
3.6.3	MOVB	19
3.6.4	PUSH	19
3.6.5	POP	19
3.6.6	ADD	20
3.6.7	SUB	20
3.6.8	AND	20
3.6.9	OR	20
3.6.10	XOR	21
3.6.11	NOT	21
3.6.12	SHR	21
3.6.13	SHL	21
3.6.14	INC	22
3.6.15	DEC	22
3.6.16	CMP	22
3.6.17	TEST	22
3.6.18	JMP	23
3.6.19	JE	23
3.6.20	JNE	23
3.6.21	CALL	23
3.6.22	RET	24
3.7	Emulator Input/Output: Sensors en DecisionOutput	24
3.8	Emulator integratie met ESF	25
3.9	Testen	26
4	Achtergrond	28
4.1	Game Programming	28

4.1.1	De game loop	28
4.1.2	Bewegende beelden	28
4.1.3	Collision detection	29
4.2	Entity system framework	29
4.2.1	Algemeen	29
4.2.2	Entity system framework concepten in opgave code	31
4.2.3	Suggestie systems en entities voor spg	31
4.2.4	Entity system framework implementatie details	36
4.3	Bibliotheken	38
4.3.1	3D grafische weergave	38
4.3.2	glm & glmc	38
4.3.3	SDL	39
4.4	Raspberry Pi & Sense HAT	40
4.4.1	Raspberry Pi	40
4.4.2	Raspberry Pi OS	40
5	Praktisch	41
5.1	Groepswork	41
5.2	Toegang GitHub Repository met opgavecode	41
5.3	Verkrijgen Raspberry Pi	41
5.4	Contact met de groepsbegeleider & Sprints	41
5.4.1	Gids voor goed gebruik issues	43
5.5	Planning	43
5.5.1	Deadlines	44
5.6	Indienen	44
5.7	Evaluatie Project	45
5.7.1	Puntenverdeling	45
5.7.2	Evaluatie Criteria	45
5.7.3	Minimumvereisten	45
5.8	Hints en opmerkingen	47
	Bijlagen	48
	Bijlage A Coördinaat stelsels	48
	Bijlage B struct Level	48
	Bijlage C Binair Level Formaat	49
	Bijlage D Raspberry Pi Getting Started	50
	Bijlage E Eenvoudig memory leaks detecteren	51
E.1	Address Sanitizer (C/C++, Linux & Mac)	51
	Bijlage F Platform-onafhankelijk programmeren	51
F.1	Build systems	51
F.2	Preprocessor macro's	52
F.3	Platformonafhankelijke interfaces	52
F.4	Platformonafhankelijke bibliotheken	52
F.5	Platformonafhankelijkheid via C standard features	52
F.6	Endian awareness	52
	Bijlage G Compileren	55

Bijlage H	Calling Conventions	56
Bijlage I	Werkverdeling	57
Bijlage J	Tips om vlot te starten	58

1 INLEIDING

Dit document bevat de strikte vereisten voor het project voor deel 1 (in Sectie 2) en voor deel 2 (in Sectie 3). Belangrijke praktische afspraken staan in Sectie 5. Relevante achtergrondinformatie, een gesuggereerde aanpak en info over de opgavecode is beschikbaar in Sectie 4.

Hiernaast krijgen jullie ook toegang tot een GitHub repository per groep. In die repository staat de startcode en extra informatie. De code die in deze repository op de “main” branch staat op het moment van de deadline wordt gezien als de indiening voor jullie groep.

1.1 ALGEMENE PROJECTINFORMATIE

Dit project loopt gedurende het volledige eerste semester en moet in een groep van twee personen uitgevoerd worden. Het project heeft als focus zoveel mogelijk aspecten van software development en systeemprogrammeren aan bod te laten komen. Hierbij denken we behalve het programmeren zelf, aan volgende aspecten: collaboration, tooling, testing, integratie van bibliotheken, documentatie en API's doornemen, bestaande code lezen en gebruiken, design en implementatie van algoritmes, etc.

Gedurende de ontwikkeling heeft elke groep een assistent die jullie zal begeleiden. Bij vragen of problemen kan die jullie verder helpen. Ook kan een meeting ingepland worden met de volledige groep.

Het einddoel van dit project is een werkende, platform-onafhankelijke C en C++ implementatie van een spel af te leveren. Er is een deadline halverwege het project, waarbij het afgewerkte C gedeelte ingediend moet worden. Het spel is een puzzel spel met isometrische afbeeldingen, genaamd “spg”. Het is met opzet vrij eenvoudig gehouden en vormt dus geen afgewerkt spel. Aan welke voorwaarden het spel precies moet voldoen wordt in de volgende secties volledig uit de doeken gedaan.

Lees ook zeker de sectie over opzetten van GitHub en indienen (Secties 5.2 en 5.6), want jullie vooruitgang wordt gecontroleerd via een GitHub-repository van UGent. Belangrijk: we gaan er van uit dat alle collaboratie en documentatie online gebeurt. Beperk dus het gebruik van online platformen waar wij geen toegang tot hebben (zoals Slack, Discord). Reden hiervoor is dat wij hier, mochten er zich groepsproblemen voordoen, geen zicht op hebben. Als je alsnog wenst gebruik te maken van dergelijke platformen, zorg dan dat de informatie en beslissingen ook via GitHub beschikbaar gemaakt worden.

Besteed voldoende tijd en aandacht aan dit project, want het gaat om 5 van de 20 punten voor het vak Systeemprogrammeren die je kan winnen, maar dus ook verliezen.

Dit project is een groepswerk en het is de bedoeling dat iedereen evenveel bijdraagt. Onder andere via GitHub zal hier over gewaakt worden, ook zal bij het verslag van deel 1 en deel 2 gevraagd worden dat elke student een score toekent aan zijn medegroepslid, die aangeeft hoeveel die heeft bijgedragen hebben aan het project. Inzet in het groepswerk zal in rekening gebracht worden bij de toegekende score.

De C11 standaard wordt gebruikt in deel 1 van het project. De C++17 standaard in deel 2. Dat betekent o.a. dat initialisatie van variabelen niet vooraan in een functie moet gebeuren. Dit is voordelig voor de leesbaarheid van de code.

We raden sterk aan gebruik te maken van de datatypes in `stdint.h`. Datatypes zoals `uint16_t` hebben een vaste lengte, terwijl het datatype `unsigned int` niet noodzakelijk op elk platform en voor elke compiler dezelfde lengte heeft.

Verder moet de code zowel op Ubuntu, als op de Raspberry Pi werken. Zie appendix G omtrent het compileren op deze platformen en appendix F omtrent platform-onafhankelijk programmeren.

Er is een gecompileerde voorbeeldoplossing gegeven voor de Raspberry Pi. Deze vinden jullie in de opgave repository (`spg_part1_piexe`, `spg_part1_edit_piexe` en `spg_part2_piexe`). Deze oplossing demonstreert alle spelregels en levels.

2 DEEL 1: OPGAVE

Het eerste deel van de opgave bestaat uit 4 delen:

- Levels inlezen en schrijven (zie Sectie 2.1)
- Spelregels implementeren (zie Sectie 2.2)
- Level Editor implementeren (zie Sectie 2.3)
- Command line argumenten ondersteunen (zie Sectie 2.4)

2.1 LEVELS OMZETTEN

spg heeft verschillende levels. Hierbij is de uiteindelijke bedoeling, om voor elk level:

- De binaire level file in te lezen en om te zetten naar de interne `Level` struct
- De interne `Level` struct in de “assemblage” om te zetten naar spel entities.
- De interne `Level` struct in de “assemblage” om te zetten naar editor entities.
- De editor entities naar een interne `Level` struct om te zetten.
- De interne `Level` struct weg te schrijven naar een binaire level file.

Er moet dus in deel 1 code geschreven worden die de levels uit de binaire level bestanden inleest en wegschrijft, om zo de `Level` structs in te vullen. Meer info over het binaire level formaat vind je in Bijlage B.

De locatie van de binaire level bestanden vind je in de opgave code terug in het bestand `game/level/fixed_level_info.c`. Voor details over het binaire level bestanden formaat, zie Bijlage C.

Meer info over het omzetten naar entities vind je in Sectie 2.2.2, en Sectie 4.2.3. Het gebruiken van het coördinatensysteem wordt beschreven in Bijlage A.

2.1.1 IMPLEMENTATIE TIP: STAPPEN TIJDELIJK OVERSLAAN

Om jullie vlot op weg te zetten, zijn behalve de binaire level bestanden, ook voor elk level gegeven:

- Een vereenvoudigde binaire level file. (Hierin komt `GM = 0x3` nooit voor. Zie Bijlage C.)
- Een stuk C code dat de `Level` struct aanmaakt (hardcoded i.p.v. dynamisch!).

Hierdoor kunnen jullie dus stappen overslaan, om zo zonder het inlezen van binaire bestanden reeds met implementeren van de assemblage en de spelregels te kunnen beginnen.

In de finale oplossing van deel 1, moeten jullie uiteraard wel alle stappen implementeren, en dus geen gebruik maken van de vereenvoudigde binaire level file of de hardcoded `Level` structs.

2.1.2 DEMO LEVEL

Er is ook een “demo level” gegeven. Dit demo level bevat de meeste spel concepten (soms op extreme manier), en kan helpen bij het testen van de code.

2.2 SPELREGELS SPG IMPLEMENTEREN A.D.H.V ENTITY SYSTEM FRAMEWORK (IN C)

Om het eigenlijke spel te implementeren, moeten `Level` structs omgezet worden naar entities en moeten de spelregels geïmplementeerd worden met behulp van een entity system framework.

2.2.1 SPELREGELS

Zoals vermeld is "spg" een eenvoudig puzzel spel met meerdere levels. Het is in elk level de bedoeling dat alle auto's de uitgang bereiken (de vlag). De levels kunnen automatisch of manueel gespeeld worden. Bij het automatisch uitspelen zoeken auto's zelf de uitgang terwijl manueel de controle aan de speler geeft.

Het eerste level vormt een inleiding die de basisconcepten demonstreren, daarna volgen de eigenlijke levels van het spel. Auto's bevinden zich op de weg en kunnen niet op het gras rijden. De speler kan een auto controleren door met de linkermuisknop een auto te selecteren en daarna gebruik te maken van de toetsen    . Er kunnen meerdere uitgangen zijn, in dat geval moet om het even welke uitgang bereikt worden.



Figuur 1: Spelconcepten

Elk level is rechthoekig en bestaat uit rijen en kolommen van cellen. Elk van deze vierkante cellen is even groot en is één van meerdere mogelijke types. Zo zijn er lege cellen waar de speler over kan lopen en obstakel cellen (muren, gebouwen, water, planten, ...) waar de speler omheen moet. Cellen kunnen vaste objecten bevatten zoals filters. Ze kunnen ook verplaatsbare objecten bevatten, zoals crates en de speler zelf.

De uitgangen zijn versperd door filters die enkel een auto doorlaten als deze genoeg crates verzameld heeft. Crates kunnen verzameld worden wanneer een auto onder een crate staat en deze nog niet een andere auto volgt. Bij manuele controle kan de speler gebruik maken van  (spatie).  (backspace) wordt gebruikt om de laatst verzamelde crate te laten vallen. Deze kan dan terug opgeraapt worden door een andere auto.

Markers helpen de auto's om automatisch het level uit te spelen en hebben geen interactie met andere objecten in het level.

Voor het gemak binnen het project, zijn enkele extra toetsen ondersteund:

-  Herstart het huidige level.
-  Ga onmiddellijk naar het volgende level.
-  of  Verlaat het spel.
-  Pauzeer het spel.
-  Toggle automatische modus waar de auto's vanzelf het level proberen uit te spelen.
-  Voert één iteratie uit van de automatische modus. De auto's voeren dus hun volgende beweging uit en wachten op verdere user input. Deze toets heeft geen effect wanneer automatische modus () reeds actief is.

2.2.2 ENTITY SYSTEM FRAMEWORK

Het spel dient geïmplementeerd te worden a.d.h.v. een entity system framework.

In de opgavecode is reeds een implementatie van een entity system framework gegeven, en is hier gebruik van makende reeds een deel van het spel geïmplementeerd.

Sectie 4.2 bevat informatie over entity system frameworks, over de implementatie die jullie als opgave meekrijgen, en over een mogelijke opdeling in systems en components die jullie kunnen maken.

We raden aan om de opsplitsing in systems en components te volgen zoals beschreven in Sectie 4.2.3. Je mag van deze opsplitsing afwijken, zolang de concepten en principes van een entity system framework gevolgd worden. Je mag de opsplitsing ook volledig zelf opnieuw maken (Let op: dit is zeer waarschijnlijk meer werk!). *Indien je niet de gesuggereerde opsplitsing volgt, dien je dit steeds in het verslag te melden en kort toe te lichten.*

2.3 LEVEL EDITOR

Een speler kan zijn eigen levels maken startend van een bestaand level. Door de editor executable uit te voeren wordt de editor modus geactiveerd en wordt een toolbar zichtbaar. Via deze toolbar kan je verschillende objecten selecteren die je op de map kan plaatsen. De toolbar selectie kan gemaakt worden met de toetsen 0 tot en met 9 alsook **shift** + 0 tot **shift** + 2. Een gele rand wijst het huidig geselecteerde object aan en kan geplaatst worden via de linkermuisknop.

Het aangepaste level kan opgeslagen worden met **F5**. Deze maakt een binaire file aan in de `levels` folder. De naam van deze file is steeds `levelfilenamebase_edited.blvl`. **F9** leest `levelfilenamebase_edited.blvl` in als deze bestaat. Indien niet, wordt `levelfilenamebase.blvl` gebruikt. Je kan dus nooit de originele level overschrijven maar leest het wel indien de edited versie niet bestaat.

De **R** toets werkt in de editor maar in tegenstelling tot **F9** laadt het steeds `levelfilenamebase.blvl`. De toetsen **N**, **P** en **K** werken niet in de editor.



Figuur 2: Level editor

De toolbar bestaat uit volgende objecten:

0 Gras	7 Herkenningpunt / Marker
1 Weg	8 Flora
2 Water	9 Filter
3 Gebouw	Shift + 0 Roteer object
4 Auto	Shift + 1 Verander kleur object (of model)
5 Vlag / Exit	Shift + 2 Verwijder object
6 Doos / Crate	

Indien de gebruiker een auto, vlag, crate, marker, flora of filter object wil toevoegen maar deze is reeds aanwezig in het level, moet de bestaande entity aangepast worden. Bij een auto zal de kleur en het model veranderen, de vlag draait zijn orientatie, de crate en marker passen hun kleur aan, de flora verandert van model en de filter past het filter nummer aan.

Het aantal verdiepen van een gebouw kan een gebruiker aanpassen door het gebouw toolbar object te selecteren en daarna op het bestaande gebouw te klikken. Dit werkt enkel door in het midden van de 3x3 tegel te klikken. Het type en kleur van het gebouw kan je niet aanpassen sinds deze automatisch (random) beslist worden bij het inladen van het level (dit is reeds geïmplementeerd in de gegeven code).

De toolbar operaties **Shift + 0** en **Shift + 1** zijn dus niet de enige manier om rotatie en kleur/model te veranderen. Voor gebouwen en auto's is **Shift + 0** wel de enige manier om ze te draaien.

2.4 COMMAND LINE ARGUMENTS

Het spel (spg_part1 en spg_part2) moet de volgende command line arguments ondersteunen:

spg_part1 [ARG ...] [<level> ...]

ARG kan een van volgende opties zijn:

- **--help** negeert alle andere argumenten, toont een duidelijke help tekst waarin de beschikbare argumenten opgelijst zijn, en start het spel niet.
- **--** Alle volgende argumenten zijn level bestandsnamen, zelfs als het getallen zijn of het woord demo.

<level> kan een van volgende opties zijn:

- Een geldig level nummer, het level die overeenkomt met het getal wordt ingeladen. '0' is het demo level, '1' is het eerste level.
- demo speelt het demo level. Het demo argument is case insensitive (bv. dEm0 is geldig).
- Een filename, speelt het level af in de file.

<level> kan meerdere keren voorkomen. Enkel de opgegeven levels worden gespeeld.

Indien geen <level> gegeven is, worden alle niet-demo levels in volgorde afgespeeld.

Voor de editor spg_part1_edit geldt volgende argumenten:

spg_part1_edit [ARG ...] <level>

ARG kan een van volgende opties zijn:

- **--help** negeert alle andere argumenten, toont een duidelijke help tekst waarin de beschikbare argumenten opgelijst zijn, en start de editor niet.
- **--** Alle volgende argumenten zijn level bestandsnamen, zelfs als het getallen zijn of het woord demo.

<level> geeft aan welk level in de editor geladen moet worden en is een van volgende opties:

- Een geldig level nummer.
- Een filename.
- demo. Het demo argument is case insensitive (bv. dEmO is geldig).

Indien niet exact 1 <level> argument gegeven is, wordt een foutboodschap gegeven en start de editor niet.

Enkele voorbeelden:

- Het spel zal levels 1, 2 en 3 spelen:
`./spg_part1`
- Het spel zal eerst level 2 en dan level 1 spelen:
`./spg_part1 2 1`
- Speel enkel het level in het bestand `gamelevellevel_filesexample.blvl`:
`./spg_part1 gamelevellevel_filesexample.blvl`
- Edit het level met de filename 1:
`./spg_part1_edit -- 1`
- Edit het demo level:
`./spg_part1_edit demo`

2.5 TESTEN

Er zijn testen meegegeven in `tests/part1/spg_part1_tests_main.c`. Deze testen de volledige levels in 1 keer en mag je niet aanpassen, zowel de signatuur van de functies als de functie code. De testen kunnen gestart worden via de executable `spg_part1_tests`. Deze testen tonen aan of het inladen van de levels correct is.

3 DEEL 2: OPGAVE

Voor deel 2 van het spel gebruiken de Auto's niet meer het automove system maar zoeken ze de uitgang met behulp van een geëmuleerd computersysteem. Er zal een eenvoudig, niet bestaand systeem geëmuleerd worden. Deze bestaat uit een CPU, een bus, geheugen en input/output. De CPU leest **binaire instructies** uit het geheugen en voert deze uit. Data in het geheugen kan gemanipuleerd worden en via reguliere geheugenmanipulatie-instructies wordt input en output afgehandeld.

De emulator moet geïmplementeerd worden in C++. Dit moet op een correct gestructureerde, objectgeoriënteerde manier ontwikkeld worden. Deze emulator zal dan een meegegeven binaire file kunnen uitvoeren die het spel kan uitspelen.

3.1 AANGEPAST SPEL: SPG_PART2

Voor deel 2 is een aparte executable voorzien die het spel start: `spg_part2`. De gegeven `CMakeLists.txt` file (zie Appendix F.1 voor meer info) compiled en linked deze file reeds correct. Voor deel 1 wordt `game/carbrain/part1` gelinked, maar niet voor deel 2. Voor deel 2 wordt `game/carbrain/part2` gecompiled en gelinked, maar dit gebeurt niet in deel 1.

Deze aparte executable doet alles wat `spg_part1` doet, maar voor het implementeren van de functies gedefinieerd in `carbrain_es_integration.h` zal `carbrain_es_integration.cpp` ipv `carbrain_es_integration.c` gebruikt worden. Hierdoor zal het bestaande (en ongewijzigde) carbrain decision system, de cpp emulator code gebruiken om de auto's automatisch te laten rijden.

3.2 ASSEMBLER

Zoals vermeld is de kern van het geëmuleerd computersysteem de CPU, die binaire instructies uitvoert.

Programmeren in binaire CPU instructies is omslachtig, dus is er gekozen om assembler code te ondersteunen. De assembler code wordt door **de assembler** omgezet naar binaire CPU instructies. De voornaamste code van de assembler is gegeven, en moeten jullie dus zelf niet implementeren.

De gegeven assembler code zet de tekstuele assembler code om naar objecten van de deels gegeven klasse `Instruction`. Deze klasse dienen jullie aan te vullen zodat ze de voorgestelde instructies kan omzetten van en naar binaire weergave.

Alhoewel de assembler zelf niet moet geïmplementeerd worden, is het is wel zeer nuttig om de assembler syntax te kennen. Het is een makkelijke manier om de CPU instructies weer te geven.

De emulator ondersteunt een gereduceerd aantal CPU instructies. Hierbij hoort een gereduceerde assembler taal. Deze assembler taal is geïnspireerd op de x86 "AT&T syntax" (ook wel "GAS syntax" genoemd). De volwaardige "AT&T syntax" is echter veel krachtiger, en bevat o.a. ondersteuning voor strings, inline berekeningen en macro-achtige defines. De vereenvoudigde assembler in deze opgave ondersteunt deze niet.

Een voorbeeld instructie:

```
MOV %a, ($MEM_LOC)
```

Hierbij is de instructie `MOV`, `%a` het bronargument, en `($MEM_LOC)` het doelargument. Sommige instructies hebben slechts een argument en er zijn instructies zonder argument.

Naast instructies laat de assembler ook labels toe:

```
labelnaam:
```

Labels staan altijd alleen op een lijn. Ze kunnen worden gebruikt om te verwijzen naar de volgende instructie. Bijvoorbeeld op deze manier:

```
MOV $5, %a
start:
```



```
DEC %a
JNE $start
```

(Bovenstaand programma zal 4 keer d.m.v JNE naar DEC %a springen.)

Commentaar is toegelaten. Alles na // wordt door de assembler genegeerd:

```
MOV %a, ($MEM_LOC) //kopieer de inhoud van register a naar het geheugen op↔
                        adres MEM_LOC
```

Tenslotte zijn “definities” toegelaten. Deze laten toe om veel voorkomende waarden met een leesbare naam op verschillende locaties te gebruiken. Een definitie start met .equ. Een voorbeeld:

```
.equ MEM_LOC 0xF700
MOV %a, ($MEM_LOC)
```

Elke CPU instructie heeft 0, 1 of 2 argumenten. Er zijn vier types argumenten:

Directe waarde: Het direct opgeven van een waarde. De syntax hiervoor is een \$ gevolgd door ofwel een decimaal getal, ofwel door 0x gevolgd door een hexadecimaal getal.

Dit is het enige type argument dat niet overal gebruikt kan worden. Het kan enkel als bronargument gebruikt worden, nooit als doel.

Voorbeeld:

```
CMP $5, $0x0A
```

In dit voorbeeld wordt het getal 5 met het getal 10 vergeleken.

Register: Een register opgeven, waaruit waardes gelezen en geschreven kunnen worden. De syntax hiervoor is % gevolgd door de naam van het register.

Voorbeeld:

```
MOV %a, %b
```

Direct geheugenadres: Een geheugenadres opgeven. De waarde wordt via de bus opgevraagd op het opgegeven adres. De syntax hiervoor is dezelfde als voor een directe waarde, maar dan tussen haakjes.

Voor de leesbaarheid is het meestal nuttig om een voorgedefinieerde waarde als adres te gebruiken, in plaats van een getal.

Voorbeeld:

```
.equ MEM_LOC 0xF700
ADD ($0xAA10), ($MEM_LOC)
```

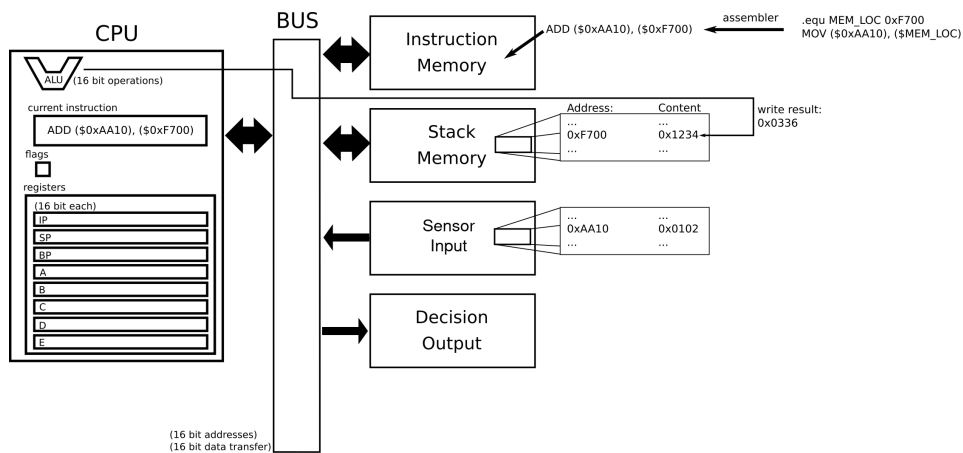
Zie ook Figuur 3.

Register met geheugenadres: Een register en optioneel een offset opgeven. Uit het register wordt de waarde gelezen. Deze wordt opgeteld bij de offset en via de bus wordt de waarde op het resulterende adres opgevraagd. De syntax hiervoor is dezelfde als voor een register, maar dan tussen haakjes. De eventuele offset staat voor de haakjes, en kan negatief zijn.

Merk op dat een offset gecombineerd met het stack pointer register zeer handig is om bepaalde waardes op de stack te lezen.

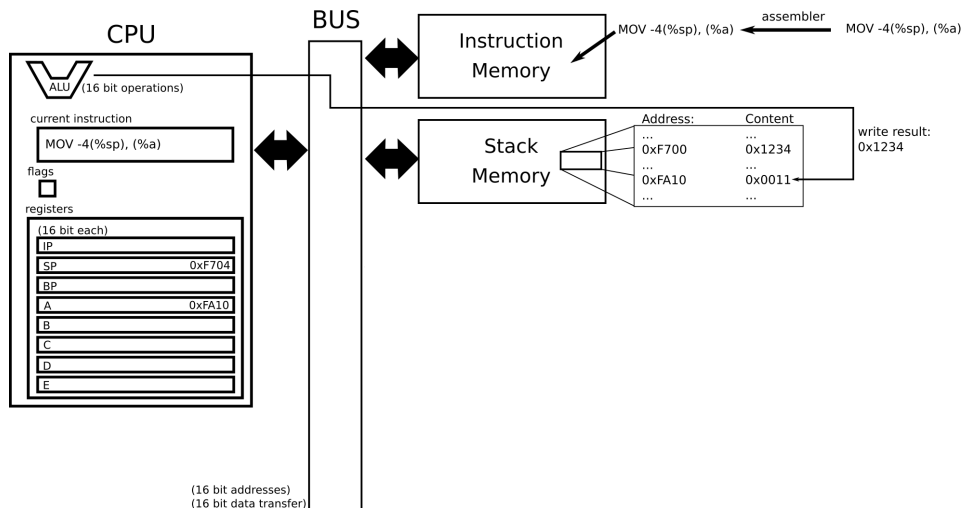
Voorbeeld:

```
MOV -4(%sp), (%a)
```



Figuur 3: Voorbeeld operatie met direct geheugenadres

In dit voorbeeld wordt 4 van de waarde van het SP (stack pointer) register afgetrokken. De resulterende waarde wordt beschouwd als een geheugenadres en de waarde op die locatie in het geheugen wordt opgevraagd. Vervolgens wordt deze gekopieerd naar het geheugen waarnaar het register "a" verwijst. Zie Figuur 4.



Figuur 4: Voorbeeld van operatie met register met geheugenadres

Merk op dat 4 bytes aftrekken van de stack pointer het geheugenadres geeft van het 2de 16-bit getal onder het getal bovenaan de stack (de stack groeit naar beneden toe). In x86 assembler gegenereerd vanuit C en C++ code, zal je dit soort geheugentoegang dikwijls zien om functie argumenten op te halen, aangezien die op de stack staan in C en C++ (bij de klassieke x86 cdecl calling convention).

3.3 EMULATOR

De emulator is een 16-bit big-endian systeem met memory-mapped I/O. De emulator bestaat uit:

- Een CPU: De CPU leest instructies via de bus (uit het geheugen) en voert deze uit. De instructie-set bevat instructies om data te verplaatsen tussen het geheugen en/of registers, om rekenkundige bewerkingen op data uit te voeren, en om de volgende uit te voeren instructie (al dan niet conditioneel) aan te passen.
- Een Bus: Deze laat de CPU toe met het geheugen, de input en de output te communiceren.
- Geheugen: 1 of meerdere blokken geheugen die beschikbaar zijn voor de CPU. In 1 van de beschikbare geheugen-blokken wordt het uit te voeren programma ingeladen. Er is ook geheugen voor de stack voorzien.

- Input: Toegang tot de sensordata van de auto's. Deze toegang gebeurt via "memory-mapped I/O", de data staat op een vaste plaats binnen het geheugenbereik waardoor de CPU de data zoals gewoon geheugen opvraagt.
- Output: Melden wat de volgende actie is die de auto moet uitvoeren. Ook hier wordt memory-mapped I/O gebruikt.

Zeer veel aspecten van realistische hardware worden buiten beschouwing gelaten bij de emulator. Zo zijn de timings van instructies en geheugentoeegang niet realistisch, is er geen user/kernel space bescherming, zijn er geen interrupts, geen conditionele jumps behalve JE en JNE, geen integer vermenigvuldiging of deling, geen floating point operaties, en heeft de assembler geen "specifiers" m.b.t. register of operatie grootte.

De hoofdklasse van de implementatie van de emulator is **Board**. Dit "bord" verbindt de verschillende onderdelen, die elk d.m.v. een specifieke klasse geïmplementeerd worden: **CPU**, **Bus**, **Memory**, **Sensors** (= input), **DecisionOutput** (= output).

De CPU kent enkel Bus, die gebruikt wordt om data te schrijven en te lezen. De CPU kan dus niet rechtstreeks communiceren met het geheugen, de input of de output. Vanuit het standpunt van de CPU is alle communicatie een aanvraag naar de bus. Meer uitleg over de CPU vind je in Sectie 3.4.

Bus houdt een lijst van "modules" bij. Elke module erft over van de "interface" **Module**. Deze interface is geïmplementeerd door **Memory**, **Sensors** en **DecisionOutput**. Bij aanvragen van de CPU zal **Bus** bepalen welke module overeenkomt met een bepaald (16-bit) geheugenadres, en de aanvraag naar de juiste module doorspelen (geheugen of input/output). Er wordt ofwel 8-bit, ofwel 16-bit per keer gelezen. Indien er fouten optreden (ongeldige locaties opgevraagd etc.) wordt een C++ exception gegoooid. Meer uitleg over **Module** en de specifieke implementaties staat in de opgave code.

Zie Sectie 3.7 voor verdere details i.v.m. de input (**Sensors**) en output (**DecisionOutput**) module en Sectie 3.8 in verband met de integratie van de emulator en het entity system framework.

De **Board** klasse wordt geïnitieerd op de volgende manier:

```
carBrainComponent->board = new Board(code_mem_size, stack_mem_size);
carBrainComponent->board->loadAndStartCodeFromExeFile(exeFilename);
```

De methode **loadAndStartCodeFromExeFile** zal volgende stappen uitvoeren:

- Check of de executable te groot is om weg te schrijven naar de code memory module.
- De instructies worden naar de juiste plaats in de juiste geheugenmodule geschreven.
- De CPU wordt geïnitieerd (zie sectie 3.4).

3.4 CPU

Zoals echte CPU's beschikt de CPU over interne opslag, in de vorm van registers. In een echte CPU zijn deze vele (honderden) malen sneller dan lezen uit het geheugen.

De emulator CPU beschikt over 8 registers van 16 bit. Deze CPU veronderstelt "big-endian" getallen bij rekenkundige bewerkingen: ADD, SUB, INC, DEC, SHR en SHL. Zie i.v.m. big-endian ook bijlage F.6.

Verder is er een speciaal "flags" register, dat info over het resultaat van de vorige instructie bijhoudt. Deze info bepaalt het al dan niet uitvoeren van conditionele sprongen.

Niet elke instructie updatet het flags register, de instructiedetails in Sectie 3.6 geven aan welke instructies dit doen.

In een echte CPU zijn er verschillende relevante bits in flags, maar bij deze emulator is dit beperkt tot één bit. Deze bit is 1 als het resultaat van de vorige instructie 0 is, en 0 als het resultaat niet 0 was. Dit laat toe de instructies JE en JNE te implementeren.

De 8 registers van de CPU zijn het IP, SP, A, B, C, D, E en F register.

Het IP register (instruction pointer) bevat steeds het geheugen-adres waar de CPU de volgende uit te voeren instructie zal ophalen. Elke keer de CPU een instructie ophaalt, zal de CPU zelf het IP register incrementeren. Bepaalde instructies (bv. JMP) zullen IP overschrijven.

Het SP register (stack pointer) bevat steeds het geheugen-adres van het huidige item in het stack geheugen. Merk op dat de stack omgekeerd gevuld wordt. Indien er iets op de stack gezet wordt (bv door PUSH), wordt SP gedecrementeerd. Indien er iets van de stack gehaald wordt (bv. door POP), wordt SP geïncrementeerd.

Merk op dat het ook mogelijk is om de SP en IP registers rechtstreeks aan te passen door ze als doel operand van een operatie te gebruiken. Je kan ze ook uitlezen door ze als bron operand van een operatie te gebruiken.

De A, B, C, D, E en F registers zijn vrij door het programma dat op de CPU draait te gebruiken. Er ligt geen betekenis vast. Deze 6 registers kunnen dus als variabele of voor het opslaan van tussentijdse rekenopdrachten gebruikt worden. Merk op dat programmas typisch ook stack geheugen voor variabelen gebruiken. Appendix H bevat info over hoe functie-argumenten typisch worden doorgegeven.

Voor het initialiseren van de emulator CPU volstaat het volgende:

- Het IP register (instruction pointer) wordt ingevuld, verwijzend naar het geheugen-adres van de eerste instructie.
- Het SP register (stack pointer) wordt gezet naar het geheugen-adres van de top (het einde) van het stack geheugen.

De methode `tick()` van CPU is het hart van de emulatie, en voert volgende stappen uit:

- De "instruction pointer" wordt gelezen, om het startadres van de volgende CPU instructie te lezen. De "instruction pointer" wordt bijgehouden in het IP register van de CPU.
- De CPU instructie wordt via de bus uit het geheugen gelezen. Instructies hebben een variabel aantal bytes en enkel de nodige bytes worden gelezen.
- De "instruction pointer" moet verwijzen naar de start van de volgende instructie. Hiervoor wordt het IP register dus aangepast.
- De instructie wordt uitgevoerd. Hiervoor moeten er mogelijks een of twee registers en/of geheugen locaties gelezen worden (via de bus). Ook is het mogelijk dat er een resultaat opgeslagen moet worden in een register, of in het geheugen. Ook zullen de interne "flags" gezet worden (er is slechts één vlag in de CPU, deze geeft aan of het resultaat van de instructie 0 is). Verder zullen sommige instructies de "instruction pointer" overschrijven (JMP, JE, JNE).
- Indien er een probleem is met de instructie, wordt een "CPUError" exception gegooid. Deze exception dient in het carbrain system van het entity system framework opgevangen te worden (in praktijk dus in `carbrain_es_integration.cpp`). Er moet een foutmelding afgeprint worden, en het spel moet vervolgens gestopt worden (`exit(1)`). In een echte CPU is er een apart mechanisme met bijvoorbeeld interrupts voorzien, maar we houden het eenvoudig in de emulator.

3.5 BINAIRE CPU INSTRUCTIES

CPU-instructies worden door de CPU uitgelezen in een binair formaat. De assembler zal dus tekstuele assembler instructies omzetten naar binaire CPU-instructies.

De CPU zal CPU-instructies uit het geheugen lezen, en ze uitvoeren.

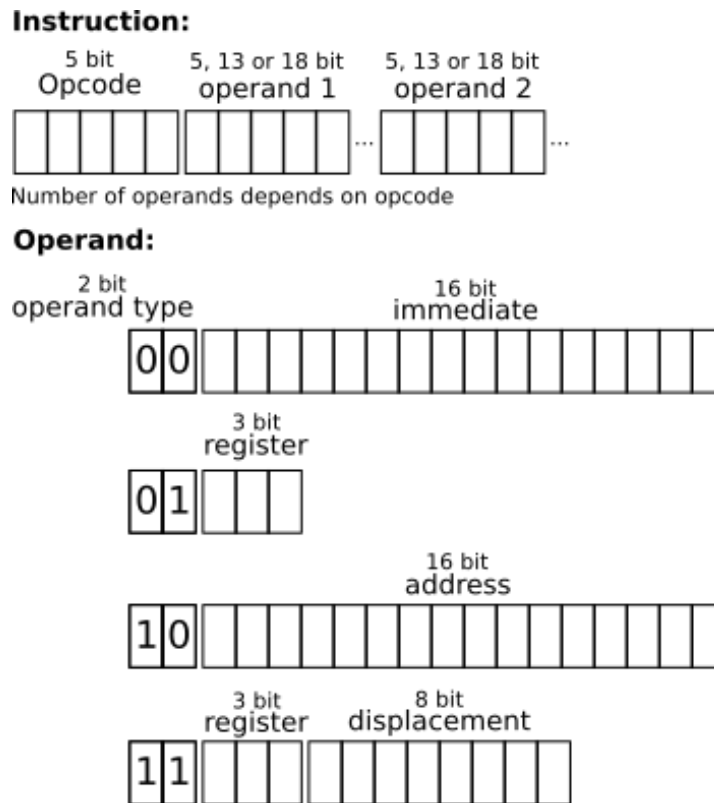
In de opgave wordt een tussenliggend formaat gebruikt: de klasse `Instruction` in de namespace `CPUInstruction`. De assembler zal tekstuele assembler instructies omzetten naar objecten van de klassen `Instruction`. Via de methode `encode` worden deze dan omgezet naar binaire instructies. De CPU zal gebruik maken van `Instruction.decode` om de binaire instructies naar objecten van de klasse `Instruction` om te zetten. De CPU kan vervolgens gebruik maken van methodes van deze objecten om eenvoudig de instructies te verwerken.

In de namespace `CPUInstruction` zijn 2 nuttige enums gedefinieerd: `Operation` en `CPUInstructionOperandType`.

3.5.1 BINAIR FORMAAT CPU INSTRUCTIES

Deze sectie beschrijft in detail het binaire formaat van de CPU instructies. In de gegeven code, is de omzetting van binaire instructies naar objecten van de klasse `Instruction` en omgekeerd reeds gegeven. Deze details zijn dus reeds geïmplementeerd. De hulpklassen die de klasse `Instruction` gebruikt bij deze omzetting, zijn onafhankelijk van deze details, maar dienen nog geïmplementeerd te worden, zie Sectie 3.5.2.

Figuur 5 geeft een overzicht van hoe instructies in binaire vorm omgezet worden.



Figuur 5: CPU instruction bit formaat.

Elke instructie bestaat uit:

Opcode (5 bits): De “operatiecode” die bepaalt welke instructie opgeslagen is.

Argumenten: Voor instructies zonder argumenten is de “opcode” het enige dat in binaire vorm opgeslagen wordt. Voor de andere instructies is er een vast aantal argumenten (1 of 2), die onmiddellijk na de opcode opgeslagen worden.

Zie Sectie 3.6 ivm de opcodes en het aantal argumenten dat elke instructie heeft.

Er zijn vier types argumenten. Elk argument wordt binair opgeslagen startende met 2 bit die het type argument bepalen:

00 = Immediate : Het argument is een direct meegegeven getal van 16 bit. De 16 bit van dit getal worden onmiddellijk na de 2 bit van het argument type opgeslagen.

01 = Register : Het argument moet uit een register gelezen worden. Het nummer van het register (3 bit) wordt onmiddellijk na de 2 bit van het argument type opgeslagen.

10 = Adres : Het argument moet uit het geheugen gehaald worden, en het geheugenadres is meegegeven. Het geheugenadres van 16 bit wordt onmiddellijk na de 2 bit van het argument type opgeslagen.

11 = Indirect Adres : Het argument moet uit het geheugen gehaald worden, en het adres zit in een register, en moet eventueel nog “opgeschoven” worden (er moet een getal bijgeteld of afgetrokken worden). Het register nummer (3 bit) waarin het geheugenadres zich bevindt wordt onmiddellijk na de 2 bit van het argument type opgeslagen. Daarna wordt een 8 bit “signed” getal opgeslagen, dat bij het adres uit het register opgeteld moet worden.

Niet elke instructie zal een veelvoud van 8 bit nodig hebben. Instructies worden altijd aangevuld met 0 bits tot het volgende veelvoud van 8 bit. Dit is de "padding" van de instructie.

Bij het opslaan van getallen van meer dan 8 bit, worden deze in "big endian" volgorde opgeslagen. Dit is logisch, omdat de getallen binnen de binaire CPU instructie sowieso niet op 8 bit grenzen opgeslagen worden, en omdat de CPU zelf "big endian" is. Bij het decoderen en encoderen van de instructies moet hier rekening mee gehouden worden, aangezien zowel de raspberry Pi als PC's "little endian" zijn. Zie ook appendix F.

Hieronder enkele voorbeelden van instructies en hun binaire representatie.

Instructie: MOV %sp, %a
Binaire: 00001 01 001 01 010 0
Hexadecimaal: 0x0A52
Details:

Opcode MOV = 1 (5 bit)
Argument Type Register = 1 (2 bit)
Register nummer %sp = 1 (3 bit)
Argument Type Register = 1 (2 bit)
Register nummer %a = 2 (3 bit)
Padding 1 bit

Instructie: ADD \$0x88, 7(%b)
Binaire: 00101 00 00000000 10001000 11 100 00000111 0000
Hexadecimaal: 0x280111C070
Details:

Opcode ADD = 5 (5 bit)
Argument Type Immediate = 0 (2 bit)
Nummer 0x0088 = 00000000 10001000 (16 bit)
Argument Type Indirect Adres = 3 (2 bit)
Register nummer %b = 4 (3 bit)
Displacement +7 (8 bit)
Padding 4 bit

Instructie: SUB \$42, (\$0x1234)
Binaire: 00110 00 0000000000101010 10 0001001000110100 0000000
Hexadecimaal: 0x300055091A00
Details:

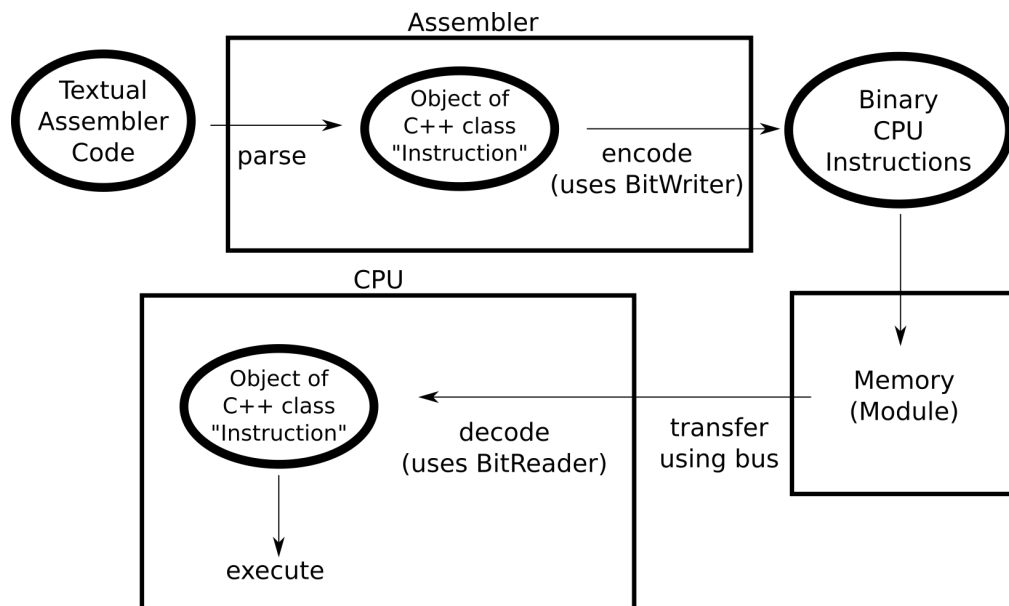
Opcode SUB = 6 (5 bit)
Argument Type Immediate = 0 (2 bit)
Nummer 42 = 00000000 00101010 (16 bit)
Argument Type Adres = 2 (2 bit)
Adres 0x1234 = 0001 0010 0011 0100 (16 bit)
Padding 7 bit

3.5.2 HULPKLASSEN VOOR HET SCHRIJVEN EN LEZEN VAN BITS

Voor het omzetten van assembler code naar binaire instructies, gebruikt de assembler de methode `encode` van de `Instruction` klasse. Deze zal bit per bit data weg schrijven.

In de implementatie van de CPU, moeten de binaire instructies uit het geheugen ingelezen worden, en omgezet worden naar de `Instruction` klasse, gebruik makende van de `decode` methode. Deze zal bit per bit lezen.

Om deze operaties te vereenvoudigen, worden twee hulpklassen gebruikt: `BitReader` en `BitWriter`. Deze dienen nog geïmplementeerd te worden.



Figuur 6: Van verwerking assembler instructies tot uitvoeren op CPU

In de opgavecode zijn de publieke methoden van deze klassen meegegeven in `game/carbrain/BitHelper.h`, samen met een beschrijving van de methodes en een voorbeeld van gebruik.

Merk op dat er gekozen wordt om getallen van meer dan 8 bit "big endian" op te slaan en te lezen.

Merk verder op dat de klasse `BitWriter` gebruik maakt van een minder gebruikt type template, geen template met type parameter, maar een template met integer parameter. Deze template parameter kan bv. worden gebruikt om in een privaat veld een array van vaste lengte te definiëren.

3.6 BESCHRIJVING CPU INSTRUCTIES

Deze sectie bevat een lijst met alle carbrain CPU instructies, en bijhorende details. Voor de betekenis van "Opcode", zie Sectie 3.5. Voor de betekenis van "Update vlag", zie Sectie 3.4.

Implementeer onderstaande CPU instructies in de `CPU` klasse. Schrijf ook testen voor elke instructie.

3.6.1 NOP

Instructie: NOP
Voluit: No Operation
Effect: Geen effect
Argumenten: 0
Opcode: 0
Update vlag: Nee

3.6.2 MOV

Instructie: MOV
Voluit: Move (Word)
Effect: Kopieert de inhoud van het bron argument, naar het doelargument. Er worden altijd 16 bits gekopieerd.
Argumenten: 2 (bron, doel)
Opcode: 1
Update vlag: Ja

3.6.3 MOVB

Instructie: MOVB
Voluit: Move Byte
Effect: Kopieert de inhoud van het bron argument, naar het doel argument. Er worden altijd 8 bits gekopieerd. Indien het doel argument een register is, worden de hoogste 8 bits op 0 gezet.
Argumenten: 2 (bron, doel)
Opcode: 2
Update vlag: Ja

Dit is de enige operatie die op slechts 1 byte uitgevoerd wordt. Alle andere operaties gebruiken steeds 2 bytes. Als het bron argument een "immediate" is, dan is deze in de binaire vorm van de instructie, zoals bij alle andere instructies, 16 bits lang (ook al wordt er slechts 8 bit gebruikt bij het uitvoeren van de instructie).

3.6.4 PUSH

Instructie: PUSH
Voluit: Push (to stack)
Effect: Trekt eerst 2 af van de inhoud van het SP register. Vervolgens wordt de inhoud van het bronargument, naar de geheugenlocatie naar waar het SP register verwijst gekopieerd.
Argumenten: 1 (bron)
Opcode: 3
Update vlag: Nee

Equivalent aan:

```
SUB 2, %sp
MOV <source>, (%sp)
```

3.6.5 POP

Instructie: POP
Voluit: Pop (from stack)
Effect: Eerst wordt de inhoud van de geheugenlocatie naar waar het SP register verwijst gekopieerd naar het doelargument. Vervolgens wordt het SP register met 2 verhoogd.
Argumenten: 1 (doel)
Opcode: 4
Update vlag: Ja, aan de hand van de uit de stack gehaalde waarde

Equivalent aan:

```
MOV (%sp), <target>
ADD 2, %sp
```

3.6.6 ADD

Instructie: ADD

Voluit: Add (Words)

Effect: Haalt bron- en doelargument op, telt ze op en slaat het resultaat op in het doelargument.

Argumenten: 2 (bron, doel)

Opcode: 5

Update vlag: Ja

MOV \$3, %a

MOV \$2, %b

ADD %a, %b

Resultaat: %a bevat nog steeds het getal 2, %b bevat het getal 5.

3.6.7 SUB

Instructie: SUB

Voluit: Subtract (Words)

Effect: Haalt bron- en doelargument op, trekt bron af van doel en slaat het resultaat op in het doelargument.

Argumenten: 2 (bron, doel)

Opcode: 6

Update vlag: Ja

Voorbeeld:

MOV \$5, %a

SUB \$3, %a

Resultaat: %a bevat het getal 2

3.6.8 AND

Instructie: AND

Voluit: (Bitwise) And

Effect: Haalt bron- en doelargument op, combineert ze met een bitwise AND operatie en slaat het resultaat op in het doelargument.

Argumenten: 2 (bron, doel)

Opcode: 7

Update vlag: Ja

3.6.9 OR

Instructie: OR

Voluit: (Bitwise) Or

Effect: Haalt bron- en doelargument op, combineert ze met een bitwise OR operatie en slaat het resultaat op in het doelargument.

Argumenten: 2 (bron, doel)

Opcode: 8

Update vlag: Ja

3.6.10 XOR

Instructie: XOR

Voluit: (Bitwise) eXclusive Or

Effect: Haalt bron- en doelargument op, combineert ze met een bitwise XOR operatie en slaat het resultaat op in het doelargument.

Argumenten: 2 (bron, doel)

Opcode: 9

Update vlag: Ja

3.6.11 NOT

Instructie: NOT

Voluit: (Bitwise) Not

Effect: Haalt het argument op, voert er een 1's complement negatie op uit (= elke 0 wordt een 1 en vice-versa), en slaat het resultaat op in het argument. Dit is equivalent aan de $\sim$ (tilde) operator.

Argumenten: 1

Opcode: 10

Update vlag: Ja

3.6.12 SHR

Instructie: SHR

Voluit: (Bit) Shift Right

Effect: Haalt "hoeveelheid" en doelargument op, schuift doel hoeveelheid bits naar rechts (aangevuld met 0), en slaat het resultaat op in het doelargument.

Argumenten: 2 (hoeveelheid, doel)

Opcode: 11

Update vlag: Ja

Voorbeeld:

```
MOV $0x000D, %a
```

```
SHR $2, %a
```

Resultaat: %a bevat het getal 3.

3.6.13 SHL

Instructie: SHL

Voluit: (Bit) Shift Left

Effect: Haalt "hoeveelheid" en doelargument op, schuift doel hoeveelheid bits naar links (aangevuld met 0), en slaat het resultaat op in het doelargument.

Argumenten: 2 (hoeveelheid, doel)

Opcode: 12

Update vlag: Ja

Voorbeeld:

```
MOV $8, %a
```

```
SHL $2, %a
```

Resultaat: %a bevat het getal 32.

3.6.14 INC

Instructie: INC

Voluit: Increment

Effect: Haalt het argument op, verhoogt het getal met 1, en slaat het resultaat op in het argument.

Argumenten: 1

Opcode: 13

Update vlag: Ja

Equivalentent aan:

ADD \$1, <target>

3.6.15 DEC

Instructie: DEC

Voluit: Decrement

Effect: Haalt het argument op, verlaagt het getal met 1, en slaat het resultaat op in het argument.

Argumenten: 1

Opcode: 14

Update vlag: Ja

Equivalentent aan:

SUB \$1, <target>

3.6.16 CMP

Instructie: CMP

Voluit: Compare

Effect: Zelfde als SUB, maar het resultaat wordt niet opgeslagen in doel. Enkel het CPU flag register wordt aangepast. Dit is nuttig om nadien een conditionele sprong uit te voeren.

Argumenten: 2 (bron, doel)

Opcode: 15

Update vlag: Ja

3.6.17 TEST

Instructie: TEST

Voluit: Test

Effect: Zelfde als AND, maar het resultaat wordt niet opgeslagen in doel. Enkel het CPU flag register wordt aangepast. Dit is nuttig om nadien een conditionele sprong uit te voeren.

Argumenten: 2 (bron, doel)

Opcode: 16

Update vlag: Ja

3.6.18 JMP

Instructie: JMP
Voluit: Jump (to address)
Effect: Onconditionele sprong. De volgende uit te voeren instructie wordt aangepast naar het argument. Dit gebeurt door %ip aan te passen.
Argumenten: 1
Opcode: 17
Update vlag: Nee

Er wordt typisch een label meegegeven. Voorbeeld (oneindige lus):

```
loop:  
ADD $1, %a  
SUB $1, %b  
JMP $loop
```

3.6.19 JE

Instructie: JE
Voluit: Jump if Equal
Effect: Conditionele sprong. Als het CPU flag register aangeeft dat het vorige resultaat 0 was, wordt de sprong uitgevoerd, anders gebeurt er niks. Om de sprong uit te voeren wordt de volgende uit te voeren instructie aangepast naar het argument. Dit gebeurt door %ip aan te passen.
Argumenten: 1
Opcode: 18
Update vlag: Nee

Opmerking: De instructie JZ (Jump if Zero) is een alias voor deze instructie. Ze doet exact hetzelfde. De reden hiervoor is de leesbaarheid van de code, na CMP is JE het meest leesbaar, maar na AND, SUB, DEC etc is JZ beter leesbaar.

3.6.20 JNE

Instructie: JNE
Voluit: Jump if Not Equal
Effect: Conditionele sprong. Als het CPU flag register aangeeft dat het vorige resultaat verschillend van 0 was, wordt de sprong uitgevoerd, anders gebeurt er niks. Om de sprong uit te voeren wordt de volgende uit te voeren instructie aangepast naar het argument. Dit gebeurt door %ip aan te passen.
Argumenten: 1
Opcode: 19
Update vlag: Nee

Opmerking: De instructie JNZ (Jump if not Zero) is een alias voor deze instructie.

3.6.21 CALL

Instructie: CALL
Voluit: Call (function)
Effect: Zet eerst de locatie van de volgende uit te voeren instructie op de stack, en past vervolgens het %ip register aan naar het meegegeven doel adres.
Argumenten: 1
Opcode: 20
Update vlag: Nee

Equivalentent aan:

```
PUSH %ip  
JMP <target>
```

3.6.22 RET

Instructie: RET

Voluit: Return (from function)

Effect: Haalt het adres van de volgende uit te voeren instructie op uit de stack, en zet het %ip register op dit adres.

Argumenten: 0

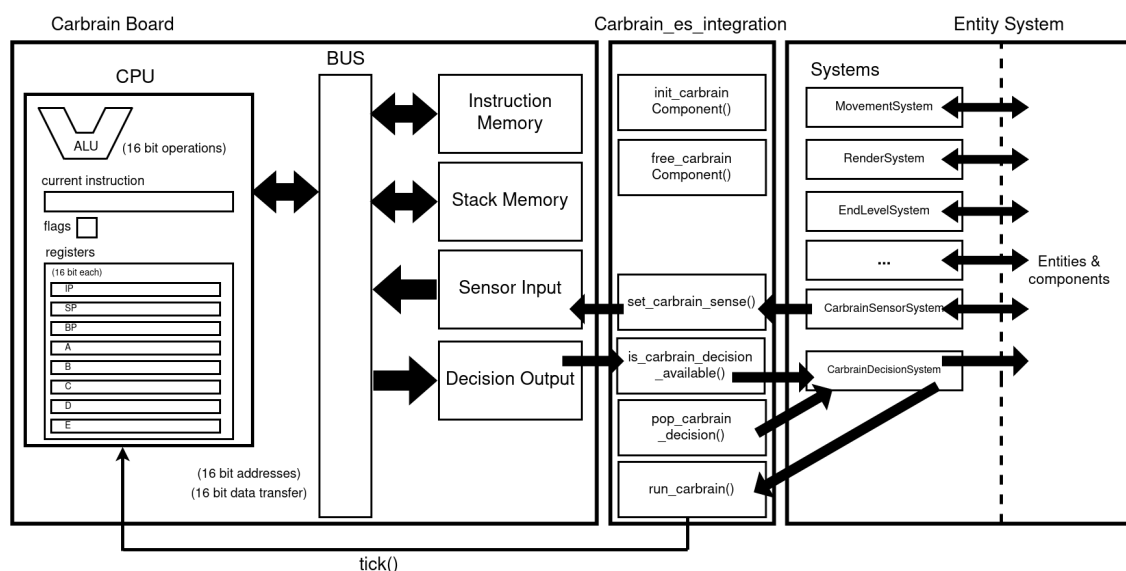
Opcode: 21

Update vlag: Nee

Equivalentent aan:

```
POP %tmp  
JMP %tmp
```

3.7 EMULATOR INPUT/OUTPUT: SENSORS EN DECISIONOUTPUT



Figuur 7: Interactie tussen de emulator en het entity system framework

De Sensors module laat programma's toe om de omgeving van de auto te bevragen.

Hiervoor biedt de module toegang tot 9 bytes. Deze komen overeen met de 8 sensor bitmask (elk 1 byte) en daarna een extra byte met het aantal crates. De beschikbare omgevingsdata is hierdoor op een vaste manier gemapt. Met behulp van aanvragen op de bus kan het programma dat op de CPU draait, opvragen wat er zich op tegels rond de auto bevindt.

De DecisionOutput module houdt bij welke beslissing(en) het carbrain programma gevraagd heeft (door via de bus naar bepaalde geheugenadressen te schrijven). Een beslissing is ofwel bewegen, ofwel draaien, ofwel een actie uitvoeren (crate opnemen of laten vallen), ofwel niets doen.

De DecisionOutput klasse heeft methodes om op te vragen of het programma reeds een beslissing heeft doorgegeven. Op die manier kan de emulator uitgevoerd worden tot een beslissing is doorgegeven door de code. Eenmaal dat gebeurd is, wordt de beslissing opgevraagd en gewist uit DecisionOutput. DecisionOutput maakt hiervoor volgende methoden beschikbaar:

`isDecisionAvailable()` geeft terug of er een beslissing beschikbaar is.

`getDecisionCount()` geeft het aantal nog uit te voeren beslissingen terug.

`popDecision()` geeft de volgende uit te voeren beslissing terug (de eerste), en verwijdert ze uit de lijst met uit te voeren beslissingen.

Als er bijvoorbeeld 2 beslissingen door de carbrain code zijn doorgegeven, stel `DECISION_RIGHT` en `DECISION_FORWARD`, dan zal `isDecisionAvailable()` "true" terug geven en zal `getDecisionCount()` 2 terug geven.

De functie `popDecision()` zal de waarde `DECISION_RIGHT` terug geven. Daarna zal `isDecisionAvailable()` "true" terug geven en zal `getDecisionCount()` 1 terug geven.

Een volgende oproep naar `popDecision()` zal de waarde `DECISION_FORWARD` terug geven waarna `isDecisionAvailable()` "false" terug zal geven en `getDecisionCount()` 0 terug zal geven.

Een carbrain programma kan acties aanvragen door bepaalde bytes (via de bus) naar de actiemodule te schrijven. De eerste byte van de `DecisionOutput` module staat initieel op 0 en bevat het aantal acties dat doorgegeven wordt. De volgende bytes zijn de gevraagde acties in volgorde.

Een carbrain programma kan dus bijvoorbeeld vragen naar links te draaien en vooruit te bewegen door middel van volgende code:

```
MOV $IO_DECISION_START, %a
MOVB $DECISION_LEFT, 1(%a)
MOVB $DECISION_FORWARD, 2(%a)
MOVB $2, (%a)
```

Merk op dat het aantal acties pas gezet wordt nadat de acties geschreven zijn (om een race-conditie te vermijden).

De interactie tussen carbrain en het entity system framework gebeurt via 2 systems, Sensor system en Decision system.

Het Sensor system bekijkt de omgeving van de auto (door het bekijken van de components van de auto entity en de andere entities in de buurt) en wordt de relevante informatie doorgegeven aan de `Sensors` module in de carbrain.

Het Decision system voert volgende stappen uit:

- Eerst wordt gekeken of de huidige actie van de auto voltooid is (door het bekijken van de components van de auto entity). De emulator wordt niet opgeroepen zolang de auto nog aan het bewegen is of een actie aan het uitvoeren is.
- De methode `tick()` van de CPU wordt (via `run_carbrain` in `carbrain_es_integration.h`) opgeroepen. `tick()` wordt herhaald tot een van volgende voorwaarden voldaan is:
 - De Decision module heeft van het programma een uit te voeren instructie ontvangen.
 - Er zijn 20 stappen uitgevoerd.
- Ten slotte wordt de `DecisionOutput` klasse geraadpleegd. Indien er minstens één uit te voeren instructie ontvangen is, wordt deze doorgegeven aan het entity system framework (door middel van het aanmaken van components).

3.8 EMULATOR INTEGRATIE MET ESF

Communicatie tussen een carbrain en het entity system framework gebeurt via de interface `carbrain_es_integration.h` zoals te zien in figuur 7. Systems kunnen van deze functies gebruik maken om te communiceren met een carbrain en dus ook een manier om vanuit C code C++ op te roepen. Deze interface wordt ofwel met een `.c` implementatie of `.cpp` gelinked afhankelijk of deel 1 of deel 2 van het project gecompileerd wordt. Bij deel 1 doen de meeste functies niets.

Merk op dat de functie `init_CarBrainComponent` opgeroepen wordt vanuit de assembler en dat het vrijgeven via `free_CarBrainComponent` gedaan wordt in `es_memory_manager.c` (dit laatste is reeds meegegeven).

3.9 TESTEN

Voor deel 2 moeten ook testen geschreven worden. Het is de bedoeling dat deze tijdens of voor het schrijven van deel 2 geschreven worden (en dan voornamelijk die voor de CPU), om het implementeren van deel 2 makkelijker te maken. De code die voor deel 2 geschreven moet worden is zeer foutgevoelig maar leent zich zeer goed tot testen, waardoor "Test Driven Development" (TDD) hier een logische keuze is. Schrijf dus eerst de testen, alvorens de code te schrijven.

Bij TDD komen de testen voor de implementatie van de code, het is een ontwikkelingsproces, geen testproces. Een robuuste manier om software componenten (units) te ontwikkelen. De ontwikkelingscyclus gaat als volgt:

1. *Tests maken*: gebaseerd op vereisten en afgesproken interfaces worden tests geschreven.
2. *Tests draaien*: de tests moeten falen in dit geval. Deze stap is erg belangrijk om zeker te zijn dat de nieuwe tests foutloos zijn.
3. *Code schrijven*: code schrijven om de tests te laten slagen. De code hoeft nog niet perfect volgens de standaard te zijn, je zal deze in een latere stap opschonen.
4. *Tests draaien*: falen de tests dan ga je terug naar stap 3, slagen ze dan kan je verder naar de volgende stap.
5. *Code herschrijven*: hier ga je de code opschonen, leesbaarder maken, etc.

Dit proces kan je blijven herhalen tot alle functionaliteit geïmplementeerd is. De tests die je schrijft zijn *unit tests*, deze beperken zich tot een kleine eenheid van je code. Aan de andere zijde van het spectrum heb je integratie testen die het volledige systeem testen, deze zijn geen onderdeel van TDD en laten we dus buiten beschouwing.

Unit tests hebben enkel betrekking op de uitvoering van hetgeen ze testen en zijn ook enkel van deze code afhankelijk. Veranderingen aan andere delen van je code mogen dus nooit deze tests doen falen. Dit maakt unit tests makkelijk onderhoudbaar en schaalbaar voor grote projecten.

Test specifiek gedrag van een eenheid in slechts één test, wanneer je dit later wil veranderen hoeft je maar enkel deze test aan te passen. Zorg er op deze manier ook voor dat je niet onnodig of dubbel test. Het nogmaals nagaan van de correcte uitvoering van een functie, die al een unit test heeft, zorgt enkel voor meer foutberichten en draagt niet bij tot de dekking van je code.

Zorg er ook voor dat je tests uitputtend zijn, en alle randgevallen en uitzonderingen opvangen. Hierbij is het dus des te belangrijk om op voorhand de vereisten uit te denken en een interface vast te leggen.

In het project kan je alle testen terugvinden in de `tests/part2` directory. Er zijn reeds enkele testen gegeven om je op weg te helpen. Het is de bedoeling de gegeven testen aan te vullen om alle nodige functionaliteit te testen. **Belangrijk**: de signatuur/interface van de code mag niet aangepast worden zodat de 2 meegegeven CPU testen niet meer compilen. Het project zal deels geëvalueerd worden door testen die de gegeven signatuur/interface volgen toe te voegen.

Testen kunnen uitgevoerd worden met de VSCode task `run spg_part2_tests (+build)`.

De testen maken gebruik van "Boost Test" (Zie o.a. https://www.boost.org/doc/libs/1_77_0/libs/test/doc/html/index.html). De Boost testen bestaan uit twee logische structuren: `BOOST_AUTO_TEST_SUITE` en `BOOST_AUTO_TEST_CASE`. Een `BOOST_AUTO_TEST_SUITE` is een logische verzameling van `BOOST_AUTO_TEST_CASE` structuren. Binnen een `BOOST_AUTO_TEST_CASE` worden de eigenlijke unit testen geschreven. Deze testen worden automatisch door het BOOST framework opgeroepen dankzij macros. De verschillende macros om vergelijkingen uit te voeren en je testen op te bouwen kan je terugvinden in de documentatie van Boost.

Je kan je eigen test files en test suites aanmaken met volgende procedure:

1. Maak een nieuwe file aan onder de `tests` directory.
2. Voeg deze code aan je nieuwe test file

```
#define __STDC_FORMAT_MACROS
#define BOOST_TEST_DYN_LINK
#include <boost/test/unit_test.hpp>
```

```

using namespace std;

BOOST_AUTO_TEST_SUITE(<TEST_SUITE_NAME>)

BOOST_AUTO_TEST_CASE(<UNIT_TEST_NAME>)
{
    <CODE>
}

BOOST_AUTO_TEST_SUITE_END()

```

3. Voer de VSCode task `cmake` uit.

4. Voer de VSCode task `run spg_part2_tests (+build)` uit.

Indien je je testen in directories wil structureren moet je de nieuwe directory ook toevoegen aan `CmakeLists.txt`. Zoek daar volgende instructie en voer je nieuwe directory eraan toe:

```

file(GLOB tests_part2_SRC
    "tests/part2/*.h"
    "tests/part2/*.cpp"
)

```

Hierna moet je opnieuw de VSCode task `cmake` uitvoeren.

4 ACHTERGROND

4.1 GAME PROGRAMMING

Spellen programmeren is op een aantal vlakken anders dan een gewone applicatie programmeren. In deze sectie worden een aantal basisaspecten van game development uit de doeken gedaan.

4.1.1 DE GAME LOOP

De centrale component van eender welk spel is de game loop. De game loop zorgt ervoor dat het spel vlot blijft lopen, onafhankelijk van het feit of de gebruiker al dan niet input genereert. Meer traditionele softwareprogramma's reageren op gebruikersinput en doen niets zonder deze input. Neem bijvoorbeeld een tekstverwerker: deze formatteert woorden en tekst terwijl de gebruiker typt. Als de gebruiker niets typt, dan doet de tekstverwerker niets. Sommige functies mogen dan wel lang duren om uit te voeren, ze zijn allemaal gestart door een gebruiker, die het programma vroeg om iets te doen.

Spellen moeten daarentegen constant blijven uitvoeren, onafhankelijk van de al dan niet aanwezige input van een gebruiker. Een game loop in pseudocode zou er zo kunnen uitzien:

```
while (user doesn't exit)
    check for user input
    move & run AI
    resolve collisions
    draw graphics & play sounds
end while
```

Hierbij wordt zolang de gebruiker het programma niet stopt, achtereenvolgens gecontroleerd welke input er gegeven is, de beweging in het spel uitgevoerd, de artificiële intelligentie (AI) van de vijand en logica van het spel uitgevoerd, worden eventuele botsingen (*collisions*) opgelost, de beelden getekend en de geluiden afgespeeld. De game loop kan verder verfijnd en aangepast worden terwijl de ontwikkeling van het spel voort gaat, maar de meeste spellen zijn op dit basisprincipe gebaseerd.

Game loops kunnen verder nog verschillen afhankelijk van het platform waarop ze ontwikkeld zijn. Als voorbeeld, een spelletje voor DOS en consoles kon alle processing resources voor zijn eigen rekening nemen en gebruiken naar wens. Een spel voor een Microsoft Windows besturingssysteem, moet uitgevoerd worden binnen de beperkingen van de process scheduler.

Verder maken de meeste huidige spellen gebruik van multi-threading, zodat bijvoorbeeld de berekening van de AI losgekoppeld kan worden van de generatie van bewegende beelden in het spel. Dit creëert uiteraard een kleine overhead, maar kan het spel vlotter doen lopen. Het zal er zeker voor zorgen dat het efficiënter kan uitgevoerd worden op hyper-threaded of multi-core processoren. Gezien de computerindustrie focust op CPU's met meerdere cores die meerdere threads parallel kunnen uitvoeren, wordt dit steeds belangrijker.

4.1.2 BEWEGENDE BEELDEN

Een ander belangrijk aspect bij games is het visuele. Er moet een vloeiend beeld weergegeven worden, wil men het spel er goed doen uitzien. De theorie rond hoeveel beelden per seconde het menselijk oog minimaal moet zien om iets als een vloeiende beweging te ervaren, is veel ingewikkelder dan meestal wordt aangenomen. Er zijn een paar vuistregels die in de meeste gevallen (blijken te) kloppen:

- Hoe meer Frames Per Seconde (FPS), hoe vloeiender het beeld.
- Hoe kleiner het verschil tussen (de beeldinformatie op) de verschillende frames, hoe vloeiender de beweging.

Dit is niet in alle gevallen correct, maar voor ons volstaat het om hier zo over te denken. Het is gemakkelijk om de snelheid van de game loop (logische lus) in frames per second (FPS) uit te drukken, op die manier is er een maat om mee te vergelijken. FPS is dan eigenlijk hoeveel keer per seconde de game state aangepast wordt.

Het aantal FPS is dus bij games enerzijds gelimiteerd door de hardware van de computer, maar ook door hoe zwaar de game loop wordt. Bij een actie ondernomen door de speler die meer berekeningen vraagt bijvoorbeeld, zal de loop-iteratie langer duren en dus het aantal zichtbare FPS tijdelijk dalen.

4.1.3 COLLISION DETECTION

Bij games wordt vaak over collision detection gesproken, het detecteren van botsingen tussen twee of meerdere objecten. In elke stap in de game loop, direct nadat een beweging is uitgevoerd door objecten, moet er vóór het hertekenen van het frame eerst gecontroleerd worden of er geen collision is, zodat er eventueel gepast op kan gereageerd worden. Dit kan bijvoorbeeld het tegen elkaar plaatsen zijn van de twee objecten in plaats van 'in' elkaar of een botsing zijn, die de objecten beiden de tegengestelde kant opstuurt. Eens deze collision gedetecteerd en opgelost is, kan het frame hertekend worden.

In een tweedimensionale ruimte valt detectie van botsingen goed mee, maar in driedimensionale ruimtes kan collision detection snel geavanceerd worden. Er zijn veel artikels en boeken geschreven rond dit onderwerp en elk van deze artikels en boeken vereist een goede wiskundige kennis.

In deze opgave is de collision detection eenvoudig gehouden, door aan te nemen dat elke vorm een "balk" ("Cuboid" in het Engels) is, met elke zijde parallel aan 1 van de assen van het coördinaatsysteem (= "uitgelijnd" tegenover de assen). Door deze vereenvoudiging, bestaat collision detection uit het nakijken van het al dan niet disjunct zijn van enkele intervallen.

Door de eenvoudige en snelle implementatie van deze methode, wordt ze in de praktijk vaak als basis collision detection gebruikt, waarna soms in meer detail naar collisions gekeken wordt. De typische term voor deze "as-uitgelijnde balken" is "Axis-aligned bounding boxes", of afgekort AABB.

Een voorbeeld:

AABB ① is volledig bepaald, door elk van volgende regels apart:

- Het middelpunt is (4, 5, 4), lengte op X-as is 4, lengte op Y-as is 9 en lengte op Z-as is 2.
- De hoekpunten zijn: (2, 1, 3) (6, 1, 3) (2, 10, 3) (6, 10, 3) (2, 1, 5) (6, 1, 5) (2, 10, 5) (6, 10, 5)
- Alle punten (x, y, z) van de AABB voldoen aan $x \in [2, 6] \wedge y \in [1, 10] \wedge z \in [3, 5]$
- Het minimale hoekpunt is (2, 1, 3) en maximale hoekpunt is (6, 10, 5)

In dit voorbeeld gebruiken we 2 extra AABB's:

- AABB ② heeft min/max hoekpunten: (0, 0, 0) en (1, 1, 5)
- AABB ③ heeft min/max hoekpunten: (4, 9, 4) en (8, 12, 10)

Om te bepalen of twee AABB's elkaar overlappen, moeten we de intervallen waarin de punten op elke as vallen bekijken. Enkel indien deze intervallen voor elk van de 3 assen overlappen, overlappen de twee AABB's elkaar.

Overlappen AABB ① en AABB ② elkaar? Op de X-as overlapt [2, 6] niet met [0, 1], op de Y-as raken [1, 10] en [0, 1], en op de Z-as overlappen [3, 5] en [0, 5]. De 2 AABB overlappen dus niet, en er is geen collision.

AABB ① en AABB ③ daarentegen overlappen in elk interval. Deze AABB's overlappen dus, en er is een collision.

Extra info is te vinden op: https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection

4.2 ENTITY SYSTEM FRAMEWORK

4.2.1 ALGEMEEN

Uit ervaring hebben we geleerd dat een klassieke objectgeoriënteerde aanpak niet altijd de beste resultaten oplevert bij het ontwerpen van een game. Abstracties die het ene moment logisch lijken, kunnen later een hindernis vormen bij het toevoegen van nieuwe functionaliteit. Voorbeelden hiervan zijn:

- Tekenend naar het scherm gebeurt door elke klasse een render-methode te laten implementeren, maar dit zorgt ervoor dat dit aspect van het spel verspreid wordt over de volledige codebase, wat het moeilijker onderhoudbaar en uitbreidbaar maakt.
- Generieke functionaliteit wordt geïmplementeerd in klassen waarvan de objecten die dit gebruiken kunnen overerven, bv. een speler erft over van de klasse InputController om te kunnen reageren op input. Maar wat als het gedrag at runtime plotseling moet veranderen op basis van de toestand? Bv. de speler dient tijdelijk te worden bestuurd door de AI voor een bepaalde cutscene (animatiefilmpje tussenin).

Bovenstaande problematiek wordt ook erkend in de game industrie en één van de voorgestelde oplossingen is het gebruik van een entity system framework. Bij deze datagedreven aanpak worden de verschillende game-objecten niet meer voorgesteld als afzonderlijke klassen, maar als generieke entiteiten die samengesteld worden aan de hand van een bepaald aantal components. Deze components bevatten heel specifieke data, die kunnen gekoppeld worden aan een game-object (bv. de positie).

Entities en hun bijhorende components zijn dus pure datacontainers en implementeren geen functionaliteit of gedrag. Dat is de verantwoordelijkheid van de verschillende systems die zullen inwerken op de geregistreerde entiteiten. Voorbeelden hiervan zijn een system om bewegingsvectoren toe te passen en een system dat entiteiten op het scherm kan tekenen.

Voor meer details en het precieze gebruik van een entity system framework, verwijzen we naar volgende artikels:

- <https://www.richardlord.net/blog/ecs/what-is-an-entity-framework>
- <https://www.richardlord.net/blog/ecs/why-use-an-entity-framework>

4.2.2 ENTITY SYSTEM FRAMEWORK CONCEPTEN IN OPGAVE CODE

Een entity system framework maakt gebruik van enkele basisconcepten, waarvan we de C implementaties binnen het project kort één per één overlopen.

COMPONENT Een component is een heel eenvoudige data-container, die data ("attributen") bevat over een entity. Sommige components bevatten zelfs geen attributen, hun al dan niet aanwezigheid bij een entiteit kan op zich al voldoende informatie zijn.

In de C-implementatie van het project is er voor elk type component een specifieke C struct, en een bijbehorende waarde in de enum `ComponentId`. (Voor components die geen data bijhouden is er enkel een waarde in de enum `ComponentId`.)

ENTITY Een entity stelt een "ding" voor in het spel. Conceptueel is een entity een samenstelling van components. In de C-implementatie bestaan entities enkel uit een unieke ID. Het geheugenbeheersysteem (Zie `es_memory_manager.h` en `es_memory_manager.c`) zal voor elke entity ID bijhouden welke specifieke components er bij horen.

SYSTEM Een system stelt een bepaald functioneel aspect van het spel voor. Bijvoorbeeld het renderen, de beweging van entities, etc. Het system bevat de code die de specifieke functionaliteit implementeert. Alle systems hebben een `system_<name>_update` functie, die door de engine in elke game loop iteratie aangeroepen wordt. In het algemeen zal elk system in elke iteratie van de game-loop specifieke entities zoeken en aanpassen.

ENGINE De Engine is het centrale onderdeel van het framework. Hier komt alles functioneel samen. Het bestaat uit een struct en enkele functies. De Engine heeft een `engine_update` functie die zal opgeroepen worden voor elke iteratie van de game loop. In deze functie wordt elk system aangeroepen. Systems zullen via het geheugenbeheersysteem entity ID's en bijbehorende components opvragen. De data van het geheugenbeheersysteem wordt in de engine struct bijgehouden.

ASSEMBLAGE Een concept dat vaak bij een entity system framework voorkomt is de "Assembly" of "Assemblage". Deze Assemblage biedt functies aan die typische soorten entities en al hun components aanmaken. Zo zal een Assemblage voor spg bijvoorbeeld een functie `create_player` en een functie `create_marker` bevatten. Een `set_level_entities(Level*)` functie is ook handig. Deze zet een ingelezen level om naar de nodige entities m.b.h.v de `create_*` functies.

In `assemblage.c` is een voorbeeld assemblage voorzien. Deze bevat ter demonstratie reeds een functie `set_level_entities`, die een level omzet in entities. De aangemaakte entities zijn in de opgave echter beperkt tot hun grafische weergave. Het is de bedoeling om in de Assemblage de `create_*` functies aan te vullen, en eventueel de assemblage uit te breiden met nuttige functies.

4.2.3 SUGGESTIE SYSTEMS EN ENTITIES VOOR SPG

De verschillende gebruikte systems en components hangen af van de functionaliteit in het spel, en van hoe gekozen wordt deze op te splitsen. De keuze van welke systems er zijn en wat ze doen, hangt zeer nauw samen met de keuze van welke components er zijn.

Wij geven verder in dit document enkele mogelijke systems en components mee. Er zijn zeker andere logische opsplitsingen mogelijk. Elke opsplitsing heeft voor- en nadelen, en er is niet altijd een duidelijke "ultieme" opsplitsing.

Jullie zijn vrij om een andere opsplitsing in systems en components te gebruiken, zo lang de opsplitsing logisch blijft en geen onnodige complexiteit toevoegt.

De systems houden zelf geen data bij, alle data zit in components. Slechts in uitzonderlijke gevallen moet een system wel data bijhouden, bijvoorbeeld als het met output en input werkt. Zo zal het Render system bijvoorbeeld data nodig hebben voor de grafische weergave. Deze wordt bijgehouden in Engine. Ook de levelloader houdt de onveranderlijke lijst met levels vast bij in engine. Voor dit project zijn dat de enige uitzonderingen. Het is niet nodig om enige andere data die een system gebruikt op te slaan buiten de components.

Merk op dat niet alle components noodzakelijk datavelden bevatten. Sommige components hebben enkel betekenis door hun al dan niet aanwezig zijn.

Ter inspiratie geven we hieronder een kort overzicht van de systems en components, die wij in onze voorbeeldoplossing gebruiken.

LEVELLOADER SYSTEM Bekijkt de globale WorldComponent en LevelInfoComponent en zorgt voor het laden van de levels. Voor het laden van de levels worden alle entities behalve WorldComponent en LevelInfoComponent eerst verwijderd, en vervolgens wordt de levelloader en assembly gebruikt om nieuwe entities voor het gewenste level toe te voegen. De levelloader zorgt er ook voor dat de editor indien nodig alle correcte informatie bevat en de editor entities ingeladen worden.

De LevelLoader system zorgt bij de editor ook voor het opslaan van de levels.

Deze LevelLoader system code is meegegeven, maar voor de editor is deze meegegeven code niet compleet.

RENDER SYSTEM Zorgt voor het renderen. Deze code is meegegeven.

Het render system zal voor elk frame de rendering voor zich nemen. Hiervoor worden alle entities bekeken die een *ArtComponent (dus CarArtComponent, GroundArtComponent, BuildingArtComponent, ColoredArtComponent of VariousArtComponent) bezitten. De positie van de weer te geven entity zit in de LocationComponent van die entity.

Verder zal het render system de camera components opzoeken en de info hieruit gebruiken, en zal het render system de "HUD" (= weergave FPS, levelnummer en editor) tekenen.

CAMERA SYSTEM Aangezien we met een isometrisch perspectief werken, wordt steeds uit dezelfde hoek gekeken. In theorie kan je de hoek wel 90 of 180 graden draaien, maar de spg engine ondersteunt dit niet. Ook in en uitzoomen wordt niet ondersteund door de engine. De "camera" bepaalt in spg dus enkel welke tegel in het midden van het scherm wordt weergegeven.

De code voor dit system is volledig meegegeven.

INPUT SYSTEM Afhandelen van toetsenbord en muis input via SDL. Dit voegt MoveRequestComponent toe wanneer pijltjestoetsen ingedrukt worden, en er nog geen MoveRequestComponent is (*Move system* verwijdert deze weer), en verwijdert MoveRequestComponent als toetsen losgelaten worden. De InputReceiver component geeft aan welke entity door toetsen aangestuurd wordt, en houdt bij welke pijltjestoetsen ingedrukt of losgelaten zijn. Via WorldComponent wordt o.a. het herstarten van een level doorgegeven.

De Input system code is deels gegeven.

MOVE SYSTEM Zoekt entities met een MoveRequestComponent, en voert de beweging uit indien ze mogelijk is (de MoveRequestComponent wordt in elk geval verwijderd). Nieuwe bewegingen zijn niet mogelijk als de entity reeds een MoveComponent heeft. Bewegingen zijn mogelijk als er op de doellocatie een entity met RoadComponent aanwezig is en er geen FilterComponent de entity blokkeert.

Algemeen zal het move system volgende logica doorlopen:

- Update de LocationComponent van alle entities met een MoveComponent.
- Verwijder MoveComponent als de move ten einde is.
- Bekijk alle entities zonder MoveComponent maar met MoveRequestComponents, en handel de MoveRequest af door een MoveComponent aan te maken.

ENDLEVEL SYSTEM Gaat na of er naar het volgend level overgaan mag worden. Zoekt entities op die ExitComponent heeft. Er wordt nagegaan of er een collision met een entity met een ExitActivatorComponent is. Wanneer alle ExitActivatorComponent entities een exit bereikt hebben, wordt het volgend level aangevraagd via de WorldComponent.

Dit gebeurt echter na enige vertraging en een animatie waar alle vlaggen een rondje draaien, die met behulp van een toegevoegde ExitedComponent beheerd wordt.

DRAGGING SYSTEM Dit systeem heeft drie verantwoordelijkheden: attachen van crates aan auto's, detachen en het bewegen van crates. De ActionAttachComponent geeft aan of een DraggerComponent (de auto) een crate wil attachen. De DraggingComponent wordt geupdate en houdt bij welke entity er meegesleept moet worden. De ActionDropComponent geeft de intentie om te detachen en dus een crate te laten vallen. De component checked alle entities of ze hun positie moeten aanpassen om de DraggerComponent entity te volgen.

Een verzameling van crates die een auto volgen kan je zien als een gelinkte lijst van elementen. De auto die aan het hoofd staat kent enkel de eerstvolgende crate, deze crate kent enkel de volgende enzovoort. De crates hebben ook een subtiële animatie die te zien is in de voorbeeld oplossing. Het is de bedoeling om dit gedrag zo dicht mogelijk te benaderen. Het doel is om een onzichtbaar touw tussen de crates te emuleren. Gesleepte crates veranderen niet van positie als ze dichterbij hun sleper zijn. Pas als ze meer dan 0.3f ver zijn, gaan ze tot maximum 0.3f getrokken worden. Dit effect is niet onmiddellijk, maar met een snelheid die groter wordt als het verder achterop geraakt.

EDIT SYSTEM Als het spel in edit modus is, zal het edit system alle EditRequestComponents afhandelen. Deze geven aan wat de speler wil veranderen aan het level en voert de nodige aanpassingen aan om het level in die staat te krijgen.

Als het spel in editor modus gestart wordt, krijgt elke positie op het grid een TileEditInfoComponent. Deze component houdt alle nodige informatie bij om een positie te beschrijven. De ground material (road, grass, water, ...) en het item (auto, gebouw, ...) die eventueel aanwezig is. De EditRequestComponent geeft de positie aan en de overeenkomstige TileEditInfoComponent wordt aangepast op basis van de geselecteerde editor tool. Merk op dat tijdens het editen van het level dit puur art components aanpast, het spel is dus niet speelbaar in editor modus.

AUTOMOVE SYSTEM (DEEL 1) Vereenvoudigde movement voor auto's. Elke auto probeert steeds een muur langs zijn rechterkant te volgen. De engine bevraagt de functie `is_carbrain_available()` om te beslissen welk system opgeroepen moet worden. Voor deel 1 zal dit steeds het automove system zijn. Merk op dat met deze vereenvoudigde movement niet alle levels uitspeelbaar zijn in automatische modus.

De rechterhand regels kunnen in 2 delen opgesplitst worden afhankelijk of de auto reeds een muur gevonden heeft aan zijn rechterkant of niet.

Indien de auto nog nooit een rechtermuur gevonden heeft gelden volgende regels:

1. Als er geen obstakel voor de auto is, bewegen we de auto naar voor.
2. Als er een obstakel voor de auto is, draaien we naar links. Er is nu dus een muur aan de rechterkant van de auto.

Indien de auto ooit een rechtermuur gevonden heeft, volgen we steeds de rechtse muur, dit komt neer op:

1. Er is een rechtermuur en we kunnen naar voor, beweeg naar voor.
2. Anders, als de auto is op een positie gekomen waar er geen rechtermuur meer is, draai naar rechts en ga vooruit.
3. Anders, als geen van vorige regels niet geldt (en er dus zowel een muur rechts als vooruit is), draai naar links. (Indien er ook links een muur is, zal door het opnieuw uitvoeren van deze 3 regels opnieuw naar links gedraaid worden.)

CARBRAINSENSOR SYSTEM (DEEL 2) Elke carbrain heeft input nodig om te beslissen wat zijn volgende beweging / actie zal zijn. Dit systeem zoekt naar specifieke entities rondom de auto en geeft deze informatie door naar het carbrain programma. Deze input wordt doorgegeven via de struct `SensorReading` die je kan terugvinden in `game/carbrain/carbrain_io.h`

Per carbrain geven we 8 posities door via bitmasks in struct `SensorReading`:

```
0
123
456
7
```

Positie 5 is de positie van de auto. De posities zijn relatief aan de locatie en richting van de auto. Als de auto draait, zal positie 2 dus de positie vlak voor de auto in de nieuwe richting zijn.

Elke positie wordt voorgesteld als een 8-bit bitmask en elke bit positie stelt een ander spelobject voor:

```
#define SENSE_ROAD 1u
#define SENSE_EXIT 2u
#define SENSE_MARKER_A 4u
#define SENSE_MARKER_B 8u
#define SENSE_MARKER_C 16u
#define SENSE_MARKER_D 32u
#define SENSE_FILTER 64u
#define SENSE_CRATE 128u
```

Zo zal een crate op de weg overeenkomen met de waarde 129 (= 128|1).

Om carbrain programmas eenvoudiger te maken, is er een bijkomende vereiste aan de bitmask: De waarde voor road wordt niet gezet, als er op de positie een filter entity aanwezig is die de huidige auto tegenhoudt. Als er bijvoorbeeld zich vlak voor een auto, die geen crates sleept, een filter met waarde 1 bevindt. Dan zal de waarde 64 gebruikt worden voor positie 2, en niet 65. Maar als er zich vlak voor een auto, die 2 crates sleept, een filter met waarde 1 bevindt. Dan zal de waarde 65 wel gebruikt worden.

Het aantal crates die de auto meesleept wordt ook doorgegeven (via `crates_dragged` in struct `SensorReading`).

Deze informatie wordt dan doorgestuurd naar de sensor module in het carbrain programma (via `set_carbrain_sense` in `carbrain_es_integration.h`).

CARBRAINDECISION SYSTEM (DEEL 2) Zoekt en voert indien nodig de volgende carbrain beslissing uit. Als een entity reeds in beweging is of een actie aan het uitvoeren is moet er niets gebeuren. Indien de auto stil staat en er geen beslissing beschikbaar is wordt het carbrain programma uitgevoerd. De carbrain beslissing wordt hierna telkens uitgevoerd.

Dit system maakt gebruik van `run_carbrain`, `is_carbrain_decision_available` en `pop_carbrain_decision` in `carbrain_es_integration.h`.

COMPONENTS IN SPG Hieronder een overzicht van alle components die wij gebruiken.

- *CameraComponent*. Info over de camera. De world entity heeft typisch deze component.
- *MoveRequestComponent*. De intentie van een entity om te bewegen.
- *MoveComponent*. Houdt de starttijd van de move bij, hoelang deze moet duren en de start- en eindlocatie.
- *LocationComponent*. De locatie van een entity in het spel.
- *DirectionComponent*. Een richting (wordt o.a. gebruikt voor de richting van bepaalde "art", zie Render system).
- *UpDownComponent*. Een richting op de Z as (wordt o.a. gebruikt voor de richting van de auto art, zie Render system).
- *ArtGround / Building / Colored / Various / CarComponent*. Render system zal elke entity die deze component en een *LocationComponent* heeft op het scherm weergeven door de nodige render calls uit te voeren.
- *InputReceiverComponent*. De entity ontvangt momenteel input van het toetsenbord.
- *InputReceiverOptionComponent*. De entity kan een input receiver worden via linker muisklik.

- *ExitComponent*. De entity is een exit. Bevat geen datavelden.
- *ExitActivatorComponent*. De entity kan een exit activeren. Bevat geen datavelden.
- *LevelEndAnimation*. Wordt toegevoegd aan een entity die een animatie moet uitvoeren wanneer het level uitge-speeld is. Houdt de timestamp bij wanneer de animatie voltooid is.
- *WorldComponent*. Houdt bij in welke staat het spel is, en welk level momenteel geladen is. Deze component wordt niet verwijderd als het volgende level geladen wordt, of een level herstart wordt.
- *LevelInfoComponent*. Houdt het level nummer en naam bij.
- *TimeComponent*. Houdt de huidige tijd bij.
- *FilterComponent*. Houdt bij hoeveel crates een auto nodig heeft om door de filter te kunnen rijden.
- *MarkerComponent*. Geeft aan dat de entity een marker is en welk kleur gebruikt moet worden door het Render system.
- *RoadComponent*. De entity is een road surface. Bevat geen datavelden.
- *TileEditInfoComponent*. Tijdens het editen van een level krijgt elke positie deze component. Deze geeft aan uit welke materiaal de grond bestaat, en of deze een item heeft. Items kunnen auto's, exits, filters, crates, decorations, buildings of markers zijn. Enkel het edit system maakt van deze component gebruik. De info in deze component is nodig voor de editor (om het level aan te passen en op te slaan), maar niet voor het spel. Bij de omzetting van het level naar entities voor het spel, gaat deze info verloren.
- *EditRequestComponent*. Geeft aan dat de user een edit wil uitvoeren op de geklikte locatie.
- *EditorToolbarComponent*. Houdt bij welke edit actie (bv. grass, road, auto, ...) actief is.
- *AutomoveControlComponent*. Toggled of de automove functionaliteit automatisch of handmatig uitgevoerd wordt. Handmatig zorgt ervoor dat alle automove entities een volgende beweging proberen uitvoeren en wachten op de volgende keypress. Bij automatische mode zal de entity direct een volgende beweging bepalen.
- *ActionAttachComponent*. De entity probeert een andere entity op te rapen. Hier zal een auto proberen een crate op te rapen.
- *ActionDropComponent*. De entity probeert een andere entity te laten vallen. Een auto kan een crate laten vallen.
- *DraggerComponent*. De entity kan andere entities die een DraggableComponent hebben dragen.
- *DraggingComponent*. Houdt bij welke entity gedragd wordt door de entity.
- *DraggableComponent*. Houdt bij welke entity gevolgd wordt en hoelang geleden de entity voor het laatst bewogen heeft.
- *CarBrainComponent*. Houdt een verwijzing naar de Board class.
- *GroundRectComponent*. Geeft een gebied aan waarover detectie van de muis gedaan wordt. De coördinaten zijn relatief aan die van LocationComponent. Als de muis over het gebied beweegt, wordt een highlight weergegeven. Het gebruik van deze component en van InputReceiverOptionComponent is reeds volledig gegeven in de opgave code.
- *FilterRequirementComponent*. Geeft aan dat een component door een filter meegeteld wordt. Een crate is de enige entity met deze component.
- *EditorInfoComponent*. Houdt info bij voor de level editor, die in het gewone spel verloren gaat. O.a. gebruikt om de afmetingen van het level en de filenaam van het level bij te houden.

4.2.4 ENTITY SYSTEM FRAMEWORK IMPLEMENTATIE DETAILS

De hoofd-documentatie over de gegeven entity system framework code is de code zelf. In praktijk is de source code vaak ook je enige documentatie.

Hieronder volgt een beknopt overzicht van de entity system framework code.

De header `es_memory_manager_collections.h` bevat allerlei handige functies om entities en components op te zoeken:

- `search_entity_1(...)` zoekt alle entities die een bepaalde component hebben. Er zijn ook versies die alle entities met 2 of 3 components zoeken (enkel entities die alle 2 of 3 gezochte components tegelijk hebben, worden terug gegeven): `search_entity_2(...)` en `search_entity_3(...)`
- `search_entity_by_location_1(...)` zoekt eveneens alle entities die een bepaalde component hebben, maar die bovendien ook binnen `max_distance` van een gevraagde positie `pos` staan (met `ignore_z` kan je de Z coördinaat eventueel negeren). Dit zoekt dus naar entities die bijkomend een "LocationComponent" met een matchende positie hebben. Er is ook een versie, die naast de positie, 2 components zoekt: `search_entity_by_location_2(...)`

Bovenstaande functies nemen als 1 van de argumenten `EntityIterator *res`. Dit is een "iterator", waarin de functie het resultaat van de zoektocht zal invullen. Er wordt via dit argument dus een *resultaat terug gegeven*, dit is een zogenaamde "output parameter"! Als je na het oproepen van bovenstaande functies `next_entity()` op de iterator oproept, bevat de iterator in het veld `entity_id` de eerste entity ID die matched. Roep je `next_entity()` nog eens op, dan bevat de iterator in het veld `entity_id` de 2de matchende entity ID. `next_entity()` geeft 1 terug als er een volgende entity was, en 0 als er geen volgende entity meer is.

Je kan dus bv alle entities met een location component zoeken op deze manier:

```
EntityIterator it;
search_entity_1(engine, COMP_LOCATION, &it);
while(next_entity(&it)) {
    EntityId entity_id = it.entity_id;
    printf("Found entity with ID %u\n", entity_id);

    LocationComponent *loc_comp =
        get_component(engine, entity_id, COMP_LOCATION);
    printf(" this entity has location=(%f,%f,%f)\n", loc_comp->pos[0],
        loc_comp->pos[1], loc_comp->pos[2]);

    if (has_component(engine, entity_id, COMP_DIRECTION)) {
        DirectionComponent *dir_comp =
            get_component(engine, entity_id, COMP_DIRECTION);
        printf(" this entity also has direction=%u\n",
            dir_comp->direction);
    }

    if (has_component(engine, entity_id, COMP_XXXXXXX)) {
        printf(" this entity also has component xxxxxx\n");
    }
}
```

Volgende functies voegen niets extra van functionaliteit toe t.o.v. `search_entity_1(...)`, maar werken zonder iterator, en zijn daardoor handig in gebruik:

- `search_first_entity_1(...)` doet hetzelfde als `search_entity_1(...)` maar geeft enkel de eerst gevonden entity ID terug. Als er geen entity gevonden is, geeft deze functie `NO_ENTITY` terug.

- `search_first_component()` doet hetzelfde als `search_entity_1(...)` maar geeft een pointer naar de gezochte *component* van de eerst gevonden entity terug, of NULL als er geen entity is gevonden.

Voorbeelden:

```
WorldComponent* world =
    search_first_component(engine, COMP_WORLD);

EntityId input_recv_entity_id =
    search_first_entity_1(engine, COMP_INPUTRECEIVER);
if (input_recv_entity_id == NO_ENTITY) ...
```

`search_entity_by_location_*` is intern geïmplementeerd m.b.h.v. een cache. Deze `gridloc_cache` is een deel van de interne werking van `es_memory_manager_collections.h`, en mag je grotendeels negeren. Het zorgt dat de `search_entity_by_location_*` lookups snel gebeuren.

Enkel wanneer je een location component aanpast moet je er rekening mee houden dat je ook de cache moet updaten via `update_gridloc_cache`. Wanneer je de cache update, moet je er ook rekening mee houden dat alle iterators reeds teruggegeven door `search_entity_by_location_*` ongeldig worden! Update de cache dus niet in de while loop van zo'n iterator. `update_gridloc_cache` roep je typisch enkel op in het move system. In de opgave code is dit dan ook vermeld in `move_system.c`.

Als je een `LocationComponent` aanpast, moet je dus `update_gridloc_cache()` oproepen, waarbij volgende regels gelden: - Je moet `update_gridloc_cache()` oproepen tussen het aanpassen van `LocationComponent` en de daarop volgende oproep van `search_entity_by_location_*`. Je kan dus eventueel verschillende aanpassingen doen, als je ondertussen `search_entity_by_location_*` niet gebruikt, of een iterator van `search_entity_by_location_*` gebruikt. - Je mag nooit `update_gridloc_cache()` oproepen vanuit de while-loop van een iterator die je van `search_entity_by_location_*` kreeg.

Merk op dat de cache enkel invloed heeft op `search_entity_by_location_*`. Andere "searches" werken correct als de cache niet up to date is.

Verder is het niet zo erg om verschillende keren in het move system de `update_gridloc_cache()` op te roepen. Zolang je het maar 1 keer doet per aanpassing van een `LocationComponent`, valt de performantie impact normaal wel mee.

4.3 BIBLIOTHEKEN

Een aantal hulpklassen zijn gegeven en maken het project eenvoudiger. Er worden ook enkele bibliotheken gebruikt. Met sommige daarvan moet geen rekening gehouden worden in het project, ze zijn afgeschermd door hulpklassen. Andere bibliotheken moeten in beperkte mate gebruikt worden.

De gebruikte externe bibliotheken zorgen er in hoofdzaak voor dat het eenvoudiger wordt om op een platformonafhankelijke manier een venster met 3D weergave te openen en om de toetsenbord- en muisinput te lezen. Een belangrijk deel van software-ontwikkeling is het integreren en gebruiken van bestaande bibliotheken. Het heeft geen zin om telkens het wiel opnieuw te gaan uitvinden, zeker niet als iemand anders gespecialiseerd is in wielen maken.

Wij leveren zelf ook een bibliotheek aan om grafische weergave te vereenvoudigen (zie Sectie 4.3.1).

De gebruikte externe bibliotheken zijn:

- SDL: Deze C-bibliotheek laat toe om op een platformonafhankelijke manier een venster te openen, de muis- en toetsenbordinput te lezen, en OpenGL te initialiseren. Zie Sectie 4.3.3.
- glm & glmc: glm (C++) & glmc (C) bevatten datastructuren voor vectoren en matrices. Het kan handig zijn hier gebruik van te maken. Zie Sectie 4.3.2 voor meer detail.
- Boost: Een verzameling van meer dan 140 diverse C++ bibliotheken, toepasbaar in een breed spectrum van applicaties. Boost¹ is nauw verbonden met het C++ standaardisatie proces, zo was er voor C++11 een Boost.SmartPtr bibliotheek. Een tiental Boost bibliotheken zijn opgenomen in het technische rapport van de *C++ Standards Committee* en de C++11 standaard. Ondertussen zijn er ook een aantal bibliotheken toegevoegd aan de C++17 standaard (zoals Boost.Variant).

4.3.1 3D GRAFISCHE WEERGAVE

De grafische kant van het spel maakt gebruik van een 2D isometrische grafische engine. Onderliggend gebruikt deze engine 3D coördinaten, die automatisch omgezet worden.

De functies voor grafische weergave bevinden zich in `game/graphics:renderer_3d_begin_draw,renderer_3d_ground_draw,renderer_3d_building_draw`, etc.

Deze worden reeds gebruikt in het render system dat voor deel 1 is meegegeven in de opgave code. Hierdoor is het niet nodig om details van de grafische engine te kennen.

4.3.2 GLM & GLMC

De glm (C++) en glmc (C) bibliotheken definiëren datastructuren voor vectoren en matrices, en implementeren operaties erop.

De glmc bibliotheek, wordt o.a. gebruikt in de (gegeven) bibliotheek voor grafische weergave. Het kan handig zijn om ze ook op andere plaatsen in het project te gebruiken.

glmc is gebaseerd op de C++ glm bibliotheek. In vergelijking met de C++ versie, is de C versie erg beperkt in functionaliteit en minder handig in gebruik. De syntax van de C++ glm bibliotheek is op zijn beurt gebaseerd op GLSL, de "Shader language" van OpenGL. Vectoren en matrices zijn in deze taal zeer makkelijk te gebruiken. Het is voor het project niet nodig om alle details van GLSL, de C++ glm bibliotheek of glmc te kennen.

De opgavecode bevat een voorbeeld van het gebruik van de glmc en glm datatypes en functies, zie `glmc_demo_main.c` en `glm_demo_main.cpp`.

In de voorbeeldoplossing worden volgende glmc datatypes vaak gebruikt:

- `t_vec3`: "vector", die 3 floats bevat. Typisch gebruikt voor X, Y en Z coördinaten.

¹<https://www.boost.org/>

- `t_ivec3`: "vector", die 3 integers bevat.

In deel 2 van de voorbeeldoplossing wordt volgende glm datatype gebruikt:

- `glm::ivec2`: "vector", die 2 integers bevat.

De file `util/glm_helper.h` bevat enkele glm hulpfuncties, o.a. om glmc vectors naar glm vectors om ze zetten.

4.3.3 SDL

De "Simple DirectMedia Layer" bibliotheek laat toe om op verschillende platformen audio, toetsenbord, muis, joystick en grafische hardware (via OpenGL) op een gelijkaardige manier aan te spreken. We maken binnen het project gebruik van SDL 2.0. Documentatie hierover vind je op <https://wiki.libsdl.org/APIByCategory>.

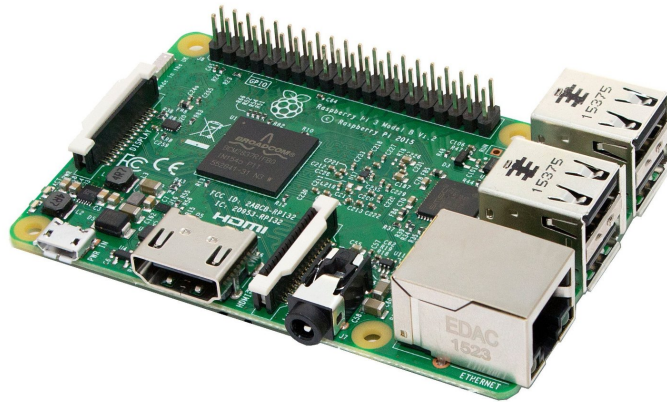
Binnen het project moet enkel gebruik worden gemaakt van SDL voor het afhandelen van input komende van muis en toetsenbord. De code om SDL te initialiseren en om een venster te openen met SDL wordt reeds afgehandeld in de gegeven code in `opengl_game_renderer.c`.

Het opgavebestand `input_system.c` bevat reeds code voor alle SDL functionaliteit, die relevant is binnen het project.

4.4 RASPBERRY PI & SENSE HAT

4.4.1 RASPBERRY PI

Voor het project zijn verschillende type Raspberry Pi beschikbaar: Raspberry Pi 3B, 3B+ en 4B.



Figuur 8: De Raspberry Pi 3.

De Raspberry Pi 3 is een single board computer gemaakt voor educatieve doeleinden. Dit bordje heeft naast een moederbord, CPU, GPU en RAM geheugen ook een chip, die 802.11n 2,4 GHz WIFI en Bluetooth 4.1 Low Energy aanbiedt. Daarnaast heeft de Pi diverse connectoren zoals een ethernet poort, USB poorten en een HDMI poort.

De Raspberry Pi 4 is een nieuwere versie, met onder andere meer geheugen, en een mini HDMI poort i.p.v. een gewone HDMI poort.

Meer informatie vind je op de Raspberry Pi website ².

4.4.2 RASPBERRY PI OS

Raspberry Pi OS (vroeger Raspbian Linux genaamd) is een derivaat van Debian Linux, dat speciaal gemaakt is om op de Raspberry Pi te draaien. Raspberry Pi OS heeft een hoop handige educatieve tools en wordt steeds verbeterd. Raspberry Pi OS, Debian en Ubuntu zijn alle drie Linux distributies van de Debian familie. Deze lijken dus heel hard op elkaar. Instructies voor Debian en Ubuntu werken meestal ook voor Raspbian.

²<https://www.raspberrypi.org/>

5 PRAKTISCH

5.1 GROEPSWERK

Het project wordt gemaakt in groepen van twee studenten. Het is verplicht hiervoor een groep op Ufora te kiezen. Je kan het Ufora forum gebruiken om een partner te zoeken. Indien je geen partner vindt, neem dan in de eerste week van het project zo snel mogelijk contact op met de begeleiders door een email te sturen naar sysprog@lists.ugent.be.

Het is de bedoeling dat jullie samenwerken en de werklast eerlijk verdelen. Meld problemen rond samenwerking altijd zo snel mogelijk aan jullie begeleider. Wacht zeker niet tot vlak voor de deadline! Indien problemen op tijd gemeld zijn kunnen we eventueel ingrijpen. In sommige situaties kunnen we bijvoorbeeld de werklast verminderen.

5.2 TOEGANG GITHUB REPOSITORY MET OPGAVECODE

Aan de hand van jullie inschrijving in een projectgroep of Ufora, wordt er automatisch een private github repository voor elke groep aangemaakt, waartoe beide groepsleden toegang krijgen. Ook jullie groepsbegeleider heeft volledige toegang tot deze repository. Deze repository wordt gebruikt als centraal samenwerkingspunt.

Lees het README van de repository voor een uitleg van wat er allemaal te vinden is in de repository.

5.3 VERKRIJGEN RASPBERRY PI

Tijdens practicum 1 krijgt iedereen een Raspberry Pi ter beschikking.

Wie niet aanwezig kon zijn, dient zijn groepsbegeleider te contacteren, om af te spreken wanneer de Raspberry Pi (in het iGent gebouw) afgehaald kan worden.

We rekenen er op dat jullie zorg dragen voor jullie Raspberry Pi.

In Appendix D vinden jullie meer info over de Raspberry Pi.

Na de laatste deadline van het project verwachten we de Raspberry Pi's terug. Jullie kunnen de Pi en toebehoren afgeven:

- Tijdens het on-campus examen.
- Op het secretariaat van iGent (verdieping 10), verdere details hierover zullen na het project op Ufora geplaatst worden.
- Na afspraak met jullie groepsbegeleider.

5.4 CONTACT MET DE GROEPSBEGELEIDER & SPRINTS

Contact met je begeleider gaat via je groepsrepository in de UGent GitHub. Om een vraag te stellen aan je begeleider maak je daar een issue aan. Maak gebruik van “@mention”³ om de aandacht van de groepsbegeleider te vragen.

We verwachten dat jullie het project werk plannen in “sprints” van 1 week. Iedere sprint begint op maandag. Elke keer een sprint start, doen jullie het volgende:

- Eerst plannen jullie de sprint. Deze planning bevat:
 - Wat jullie de volgende week zullen implementeren.
 - Hoe jullie dit werk zullen verdelen.
- Vervolgens rapporteren jullie deze planning door naar jullie begeleider:
 - Maak in jullie GitHub repository een “Milestone” aan, met als naam “Sprint NR” (met “NR” het volgnummer van de sprint). Vergeet de einddatum niet.

³<https://guides.github.com/features/issues/>

- Maak binnen de milestone een “Issue” aan, met als naam “Sprint NR”, waarin de werkverdeling voor deze sprint kort samengevat is. Deze werkverdeling maak je zelf op. Hou hierbij rekening met hoeveel tijd je kan vrijmaken voor het vak tijdens deze sprint. Het is bijvoorbeeld mogelijk dat je tijdens bepaalde sprints (bijna) geen werk plant omdat er deadlines zijn van andere vakken. Indien jullie tijdens de sprint merken dat je teveel of te weinig werk geplanned hebt dan kan je dit ook verder bespreken in deze issue en eventueel de werkverdeling aanpassen.
 - Maak daarna in deze milestone *per gepland werk item* ook nog één issue aan. “Assign” deze issues aan de juiste persoon. In deze issues kunnen jullie bijhouden hoe ver jullie geraakt zijn voor elke onderdeel, en of er problemen zijn. Gebruik gerust “@mention” om de begeleider te vragen naar een issue te kijken om bijvoorbeeld een vraag te beantwoorden. Sluit het issue wanneer het werk klaar is.
 - De volgende week implementeren en testen jullie wat jullie gepland hebben. Via de issues in de milestone houden jullie de vooruitgang bij.
- Geef daarom ook elke commit die jullie maken een duidelijke boodschap, waarin jullie steeds het issue ID vermelden. Op die manier zal GitHub de commits automatisch aan de bijhorende issues linken. Een voorbeeld: Stel dat github issue nr 7 over het implementeren van het Movement system, en je voegt eerste code hiervoor toe. Dan neem je als commit message: “#7 eerste implementatie movement system”. Het kan ook handig zijn om voor sommige issues aparte branches en pull requests te gebruiken.
- Op het einde van elke sprint, rapporteren jullie wat jullie bereikt hebben aan de begeleider. Voeg hiervoor commentaar toe aan het “Sprint X” issue dat bij de start van de sprint werd aangemaakt. Laat hier weten of jullie meer of minder dan geplanned hebben gedaan. Vragen zijn zoals steeds ook welkom.

Voor vragen kunnen jullie steeds bij jullie groepsbegeleider terecht, dus zowel voor, tijdens en na elke sprint. Wacht niet tot het einde van een sprint om een vraag te stellen. Voor de meeste vragen zijn GitHub issues aangewezen. Voor sommige vragen kan een email beter passen. Dat is uiteraard geen probleem (zet je mede groepslid liefst in CC). Het is ook mogelijk om af te spreken met de begeleider. Voor sommige complexe vragen of code problemen, werkt dit vlotter. Neem hiervoor contact op met je begeleider via een issue of email. De ervaring leert dat genoeg vragen stellen, in welke vorm dan ook, een zeer efficiënte manier is om vlot verder te raken met het project. Aarzel dus niet vragen te stellen. Jullie vragen mogen zowel specifiek over het project gaan, over de practica, of algemeen over C/C++.

De GitHub milestone, het issue met werkverdeling, en het verslag van de vorige sprint moeten steeds ten laatste op maandagavond klaar zijn. *Er zullen punten voor het project verloren gaan indien deze rapportering niet elke week tijdig klaar is.* De planning ligt echter volledig in jullie handen. We kunnen er commentaar op geven, maar we laten jullie vrij. Hoe onverstandig het ook is, als je het elke week correct rapporteert, mag je dus plannen al het werk enkel in de week voor de deadline te doen.

Voor de eerste sprint maken we voor jullie de nodige issues en milestone reeds aan. Dit beschrijft wat we verwachten dat je doet tijdens de eerste sprint. Voor de volgende sprints zal je deze dus zelf moeten aanmaken. Ga naar de repository van je groep en open de issues om de doelen voor de eerste week te lezen.

We verwachten dat dit alles in de eerste week gebeurt, en jullie problemen snel rapporteren, zodat jullie vlot kunnen starten. Uiteraard mogen jullie ook reeds met het implementeren van het project starten. Voeg dan gewoon een issue toe aan de milestone van sprint 1, om dit te rapporteren.

Merk op dat we het eerste sprint eindverslag en het startverslag van sprint 2 verwachten op maandagavond 24 oktober.

De tijd voor het project is beperkt, dus meld problemen op tijd, en stel snel genoeg vragen. Het is zeer makkelijk om te onderschatten hoeveel werk het project is. We raden daarom zeer sterk aan om snel te starten, en niet vlak voor de deadline in actie te schieten.

5.4.1 GIDS VOOR GOED GEBRUIK ISSUES

- GitHub issues en comments maken gebruik van Markdown⁴. GitHub heeft een handige cheat sheet⁵ waarin ook "github flavored markdown" gedemonstreerd wordt.
- Indien de vraag betrekking heeft tot code, "push" die code eerst naar de repository (best in een aparte branch als de code niet werkt). In het issue link je vervolgens naar de relevante lijnen code⁶
- Voeg screenshots of screencap toe als je vragen hebt over een visueel probleem.
- Gebruik "@mention" om de aandacht van personen (typisch je begeleider) op een issue te vragen. Type het "@" teken, en je zal de github account kunnen kiezen die je wil "vermelden". Deze persoon is dan automatisch "subscribed" aan het issue, en zal typisch een email over wijzigingen in het issue ontvangen. Een voorbeeld:

Ik snap niet wanneer het kernwoord `typename` gebruikt moet worden bij C++ templates.
@bvolckae kan je dit toelichten?

- "Assign" het issue aan de relevante persoon. Dat is typisch de persoon die code moet implementeren, maar het kan ook de persoon zijn die een vraag moet beantwoorden, omdat er anders niet verder gewerkt kan worden. Je kan het issue dus ook aan je begeleider toewijzen, als je vast zit tot je antwoord op een vraag hebt. Een nieuw issue om een vraag aan je begeleider te stellen, "assign" je ook aan je begeleider.
- Gebruik geen screenshots om code in een issue op te nemen. Er zijn 2 betere manieren:
 - Je kan makkelijk verwijzen naar code uit je repository, met een handige link en lijn nummers er bij, door gebruik te maken van de link functionaliteit van github⁷.
 - Je kan ook C code die niet in je repository staat tonen met behulp van markdown code block syntax⁸.

Een voorbeeld:

```
```C
int myfunc(int arg) {
 return arg + 2;
}
```
```

5.5 PLANNING

Op elke maandagnamiddag is er ook steeds mogelijkheid om met de begeleider te overleggen (via een meeting in het iGent gebouw, of via confcall). Ook op andere momenten is dit mogelijk. Dit is steeds op voorhand af te spreken, contacteer hiervoor je begeleider.

⁴<https://daringfireball.net/projects/markdown/>

⁵<https://guides.github.com/pdfs/markdown-cheatsheet-online.pdf>

⁶<https://docs.github.com/en/github/managing-your-work-on-github/creating-a-permanent-link-to-a-code-snippet>

⁷<https://docs.github.com/en/free-pro-team@latest/github/managing-your-work-on-github/creating-a-permanent-link-to-a-code-snippet>

⁸<https://docs.github.com/en/enterprise-server@2.20/github/writing-on-github/creating-and-highlighting-code-blocks>

| | |
|----------------------|--|
| Maandag 17/10 | Practicum 1 + introductie project + start sprint 1 |
| Maandag 24/10 | Practicum 2 + einde sprint 1 + start sprint 2 |
| Maandag 31/10 | einde/start van sprint |
| Maandag 7/11 | einde/start van sprint |
| Zondag 13/11 | Deadline deel 1 |
| Maandag 14/11 | einde/start van sprint |
| Maandag 21/11 | einde/start van sprint |
| Maandag 28/11 | einde/start van sprint |
| Maandag 5/12 | einde/start van sprint |
| Zondag 11/12 | Deadline deel 2 + einde laatste sprint |

5.5.1 DEADLINES

Het project heeft twee grote deadlines. De eerste voor het afleveren van het C gedeelte, de tweede voor het afleveren van het C++ gedeelte.

Voor elke deadline dient de code, het verslag, en de peer evaluatie ingediend te worden, zie 5.6.

- Deadline C (deel 1): **Zondag 13 November 23:59**
- Deadline C++ (deel 2): **Zondag 11 December 23:59**

5.6 INDIENEN

Er dienen op elke deadline 3 zaken ingediend te worden:

- **Code:** De laatste commit op de GitHub repository (main branch) voor de deadline wordt als finale code gezien. Zorg dus dat deze versie zeker compileert en uitvoerbaar is. De code zal getest worden met de GCC 10 compiler op Ubuntu en met GCC 10 op de Raspberry Pi. Zorg dus dat je deze compiler gebruikt voor het maken van de opgave en test je code zowel op Ubuntu als op de Raspberry Pi.
- **Verslag:** Hou in de GitHub repository een kort verslag bij in het bestand `REPORT_1.md` (`REPORT_2.md` voor deel 2) (max. vier bladzijden, geen minimum) met daarin:
 - naam, voornaam en richting van de groepsleden + groepsnummer van Ufora
 - De finale taakverdeling: wie heeft wat gedaan?
 - Een kort overzicht van welke gevraagde functionaliteit niet werkt in de finale oplossing, en of er gekende bugs zijn.
 - Eventueel een extra woordje uitleg bij je ontwerpbeslissingen. Zeker als van de door ons gesuggereerde aanpak wordt afgeweken, moet in het verslag vermeld worden hoe werd afgeweken, en waarom. Dit hoeft niet uitgebreid te zijn.

Ook voor de finale versie van het verslag, wordt gekeken naar de laatste commit voor de deadline op de GitHub repository (main branch).

- **Peer Evaluatie:** Elke student stuurt ook **individueel** een mail naar Bruno Volckaert (bruno.volckaert@ugent.be) en de groepsbegeleider met daarin een "peer evaluatie" van je groepspartner.

Het onderwerp van deze mail is: "sysprog - peerevaluatie - groep <groepnummer> - <naam groepspartner>".

Ga hierbij als volgt te werk:

- Als je groepslid evenveel meegewerkt heeft aan het project dan geef je:

- * groepslid: 100%
- Heeft bijvoorbeeld je groepslid volgens jou een kwart minder bijgedragen aan de oplossing, dan geef je die persoon 75%:
- * groepslid: 75%
- Gelijkaardig mag je, als je vindt dat je groepslid meer bijgedragen heeft aan het project, die een hogere score toekennen:
- * groepslid: 125%

Hou hierbij volgende in acht:

- Je geeft jezelf geen score.
- Indien je afwijkt van de 100%, graag ook een woordje motivatie.
- Groepsleden krijgen slechts individueel punten indien er grote verschillen vastgesteld worden. Het is dus niet zo dat peer evaluatie scores altijd automatisch omgezet worden naar afwijkende punten. Ook is het niet omdat beide partners elkaar 100% toekennen, dat de begeleiders dat volgen in de score. De peer evaluatie is slechts één element waarbij rekening gehouden wordt.
- Het is niet omdat er op het einde een peer evaluatie doorgestuurd wordt, dat problemen niet eerder gemeld moeten worden! Meld problemen i.v.m. samenwerking altijd snel genoeg. Eventueel kunnen we dan ingrijpen. In sommige situaties kunnen we bijvoorbeeld de groep splitsen en/of de werklast verminderen.

5.7 EVALUATIE PROJECT

5.7.1 PUNTENVERDELING

Het project geldt voor 5 punten van het examen. Deze 5 punten kunnen als volgt verdiend worden:

- 2,5 punten op het C gedeelte.
- 2,5 punten op het C++ gedeelte.

Wanneer er een (groot) verschil in inbreng van teamleden wordt vastgesteld, zal er individueel gequoteerd worden. Hierbij zal o.a. rekening gehouden worden met de peer review door je groepspartner (zie 5.6)

5.7.2 EVALUATIE CRITERIA

Het project wordt voornamelijk geëvalueerd op welke deel-functionaliteiten correct werken. In de GitHub repository van het project staat een lijst met alle functionaliteiten waarop geëvalueerd wordt (bestanden GRADING_1.md en GRADING_2.md).

Met deze lijst kan makkelijk nagegaan worden wat er nog ontbreekt aan het project. Als alle functionaliteiten in de lijst correct voorzien zijn, leidt dit normaal tot een zeer hoge of maximale score voor het project. Er kunnen echter ook punten verloren worden door memory leaks, crashes en andere algemene fouten. Zorg ook zeker dat het project werkt op zowel PC als Raspberry Pi.

5.7.3 MINIMUMVEREISTEN

Bij het indienen van het project vereisen we dat een minimum doel bereikt is. Indien dit niet het geval is, zal een nul-score gegeven worden.

Voor beide delen van het project moet de code steeds op zowel de Raspberry Pi als op Ubuntu compileren. Er moet ook telkens een verslag aanwezig zijn.

Voor deel 1 van het project is het minimumdoel dat een auto kan geselecteerd worden, en dat deze kan bewegen met behulp van de pijltjes toetsen.

Voor deel 2 van het project is het minimumdoel dat de 2 meegegeven CPU testen compilen en succesvol uitgevoerd worden. Deze minimumdoelen zijn eenvoudig haalbaar. Merk op dat indien enkel deze minimale doelen gehaald zijn, het project wel zal verbeterd worden, maar de score zeer laag zal zijn.

5.8 HINTS EN OPMERKINGEN

- Het is toegestaan (en mogelijk soms nodig) om opgegeven header files en cpp files aan te passen.
- Begin niet onmiddellijk te programmeren. Lees eerst aandachtig de opgave en denk na hoe je de verschillende problemen zou oplossen (vb. welke klassen ga je gebruiken, wat houden ze bij, etc.).
- Het voordeel van een entity system framework is dat je bepaalde systems kan uitschakelen tijdens het debuggen door ze gewoon even niet aan de Engine toe te voegen.
- Maak in het C++ gedeelte zoveel mogelijk gebruik van de Standard Template Library (STL).

We raden o.a. sterk het gebruik van `std::shared_ptr` aan. Volgende link kan daarbij ook van pas komen:
<http://www.cppreference.com/wiki/stl/start>

- Alle opmerkingen bij de oefenset-opgaves gelden ook hier: wees zuinig met geheugen, let op voor dangling pointers, geef alles wat gealloceerd wordt ook weer vrij, controleer of je open bestanden wel correct geopend zijn en gesloten worden, bescherm je header files, etc.
- Als referentiecompiler wordt GCC 10 gebruikt (en GCC 10 op Raspberry pi). Zorg er dus voor dat je project daarmee compileert en uitvoerbaar is!
- Probeer zoveel mogelijk compiler warnings te vermijden; deze duiden meestal op fouten die de compiler voor jou zal oplossen (al dan niet op een correcte manier), maar die je evengoed kan vermijden door je code aan te passen.
- Vanzelfsprekend zijn er zaken waar de opgave je vrij in laat; deze mag je zelf kiezen naar eigen inzicht. Waarschijnlijk is het interessant dit in het verslag van het project toe te lichten. Verduidelijk in alle geval je broncode met commentaar waar dat nuttig is!

Veel succes!

Bijlagen

BIJLAGE A COÖRDINAAT STELSELS



Figuur 9: De 2 coördinaat stelsels

Het spel en de engine gebruiken verschillende coördinaat stelsels, waartussen soms omgezet moet worden.

- In de level bestanden, is elke isometrische tegel 3 eenheden breed en diep. Elke “level tegel” heeft een discrete X en Y coördinaat. Zie Figuur 9a. Je gebruikt dit coördinaat stelsel als je met de level files en de bijhorende (struct Level) werkt.
- In de spel 3D engine, is elke isometrische tegel 1 eenheid breed en diep. De coördinaten op de assen komen overeen met de hoekpunten van de isometrische tegel. De 3D engine slaat coördinaten op als `float`, en coördinaten hoeven dus geen discrete getallen te zijn. De auto bevindt zich dus in de level tegel met 3D engine X coördinaten tussen $[0.333..., 0.666...]$ en Y coördinaten tussen $[0.333..., 0.666...]$. Het middelpunt van de auto staat op het middelpunt van de isometrische tegel, dus op het 3D engine punt $(0.5, 0.5)$. Zie Figuur 9b. Je gebruikt dit coördinaat stelsel in entity system framework, specifiek in de component Location.

De omzetting tussen beide coördinaatstelsels gebeurt in de gegeven code reeds in de assemblage (zie gebruik van `level_pos_to_entity_pos`).

BIJLAGE B STRUCT LEVEL

In `game/level/level.h` wordt de `Level` struct gedefinieerd. Deze bevat de basis info over een level, die door de assemblage moet omgezet worden naar entities en components.

De struct is als volgt gedefinieerd:

```
typedef struct Level {
    char* name;

    unsigned int width;
    unsigned int height;
    LevelPoint** points;

    LevelItem* items;
    size_t item_count;
} Level;
```

`points` is een tweedimensionale array, met als eerst index de x positie (steeds tussen 0 en `width-1`) en als tweede index de y positie (steeds tussen 0 en `height-1`).

`items` is een array met `item_count` items.

In `level_init` zal je voor `points` en `items` dus dynamisch geheugen alloceren zodat de nodige punten en items opgeslagen kunnen worden. Ook voor `name` zal dynamisch geheugen gebruikt worden, om een kopie van de gegeven

`level_free` zal al dit dynamische geheugen weer vrijgeven. Zorg er voor dat `level_free` niet opgeroepen wordt voor de hardcoded levels (of dat level free niets doet voor de hardcoded levels). De hardcoded levels maken namelijk geen gebruik van `level_init` en gebruiken ook geen dynamisch geheugen.

| Byte Offset | 0 | 1 | 2 | 3 |
|-------------|------------------------|---------|----------------------|---------------------------------|
| 0 | Header (6B) | | | |
| 4 | | | Version (2B) | |
| 8 | Width (2B) | | Height (2B) | |
| 12 | Level_name_length (1B) | | Level_name (? B) ... | |
| Tiles | GM (2b) | GH (2b) | GM (0x3) ? GM (2b) | Padding |
| Items(1) | Item_count (2B) | | Item_X_coord (2B) | |
| Items(2) | Item_Y_coord (2B) | | LIT (3b) | VAR (+ Padding) (3-5b) (+ 0-5b) |

- **Header - 6 bytes:** De eerste 8 bytes bevatten een identificatie van het bestandsformaat. De eerste 6 bytes hiervan zijn de vaste 6 tekens (in ASCII coding): "SPGbin".
- **Version - 2 bytes:** Deze 2 bytes van zijn ook deel van de identificatie van het bestandsformaat en bevatten het versie nummer van het spel. In dit geval moet dit steeds overeen komen met 2021. Dit getal wordt weggeschreven in big endian format.
- **Width - 2 bytes:** De breedte van het speelveld. Dit getal wordt weggeschreven in big endian format.
- **Height - 2 bytes:** De hoogte van het speelveld. Dit getal wordt weggeschreven in big endian format.
- **Level_name_length - 1 byte:** Geeft weer hoeveel bytes de level naam bevat.
- **Level_name:** De naam van het level. Zonder \0 byte op het einde.
- **Tiles:** Alle posities worden vervolgens 1-na-1 weggeschreven, in het opgegeven formaat. De volgorde is steeds "rij eerst", dus eerst positie $[x=0, y=0]$, dan $[x=0, y=1]$, en pas na de volledige eerste rij, positie $[x=1, y=0]$, ... In het geval dat de tiles eindigen in een niet volledige byte wordt padding toegevoegd met de waarde 0.
- **GM - 2 bits:** Ground Material beschrijft het grondtype van de positie. De waarde voor elk type wordt bepaald door enum `Material`. Een waarde van 0x3 geeft aan dat deze positie dezelfde is als de vorige en volgt er geen GH. Deze regel zorgt ervoor dat de bestandsgrootte van de levels veel kleiner kan uitvallen. Bij de opgave zijn er level bestanden meegegeven met en zonder deze optimalisatie regel.

- 0x0 -> ART GRASS

- 0x1 -> ART_WATER
 - 0x2 -> ART_ROAD
 - 0x3 -> Herhaling vorige positie
- **GH - 2 bits:** Ground Height geeft weer op welke plaats op de z-as de tegel staat. Ground height kan in het spel een waarde van 1 tot en met 4 hebben. De 2 bits in het GH veld kunnen een waarde hebben van 0 tot en met 3. De waarde in de binaire level file is daarom steeds 1 lager dan de echte waarde (dus: echte ground height waarde = GH veld + 1).
 - **Items:** Na de posities volgt de item count, en vervolgens alle items 1-na-1, steeds in het beschreven formaat. Als de item count 0 is, volgen er uiteraard geen items.
 - **Item_count - 2 bytes:** Geeft weer hoeveel items er zijn. Dit getal wordt weggeschreven in big endian format, en kan niet negatief zijn. Het kan wel 0 zijn.
 - **Item_X_coord - 2 bytes:** De X coördinaat van het item. Dit getal wordt weggeschreven in big endian format. Dit getal kan niet negatief zijn.
 - **Item_Y_coord - 2 bytes:** De Y coördinaat van het item. Dit getal wordt weggeschreven in big endian format. Dit getal kan niet negatief zijn.
 - **LIT - 3 bits:** Level Item Type geeft het type item aan en wordt gebruikt om het veld VAR (+ Padding) te interpreteren. Zie `LevelItemType` in `level.h`.
 - **VAR (+ Padding) - 5 bits:** dit veld bestaat steeds uit 5 bits. Deze moeten afhankelijk van het LIT veld anders verwerkt worden. Indien niet de volle 5 bits gebruikt worden, wordt bij dit veld een padding ingevuld (waarde 0) tot alle 5 bits in gebruikt zijn.

| LIT | VAR highest bits | VAR lowest bits |
|------------|--------------------|------------------|
| CAR | 2 bits direction | 3 bits color |
| EXIT | 2 bits direction | 3 bits 0-padding |
| FILTER | 3 bits count | 2 bits 0-padding |
| MARKER | 3 bits color | 2 bits 0-padding |
| CRATE | 3 bits color | 2 bits 0-padding |
| DECORATION | 2 bits art variant | 3 bits 0-padding |
| BUILDING | 3 bits count | 2 bits direction |

BIJLAGE D RASPBERRY PI GETTING STARTED

In de online documentatie vind je de getting started guide voor de Raspberry Pi: <https://github.ugent.be/Systeemprogrammeren/docs/blob/master/tutorial-pi.md>

BIJLAGE E EENVOUDIG MEMORY LEAKS DETECTEREN

E.1 ADDRESS SANITIZER (C/C++, LINUX & MAC)

Op Linux en Mac is het heel eenvoudig om memory leaks en andere geheugenfouten zoals een “double free” op te sporen door middel van AddressSanitizer (ASan). Dit is een geheugen foutdetector origineel ontwikkeld door Google voor de Clang compiler die sinds versie 4.8 ook in GCC verwerkt zit. Deze staat op Mac en Linux automatisch aan voor het project. Je kan die op andere projecten manueel aanzetten door de `-fsanitize=address -fno-omit-frame-pointer` flags mee te geven met de compiler en de linker.

De compiler linkt alle geheugenoperaties zoals `malloc` en `free` door naar ASan, zodat deze alle geheugenoperaties kan bijhouden. Bij het afsluiten van het programma wordt er gekeken welk geheugen er aangevraagd is zonder het te verwijderen en rapporteert ASan alle memory leaks in het formaat dat hieronder te zien is.

```
=====
==22505==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 20000 byte(s) in 1 object(s) allocated from:
#0 0x7f9ad892a848 in __interceptor_malloc (/usr/lib/x86_64-linux-gnu/libasan.so.5+0xee848)
#1 0x55b9aede4800 in graphics_init /sysprog/game/graphics/opengl_game_renderer.c:364
#2 0x55b9aede4605 in graphics_alloc /sysprog/game/graphics/opengl_game_renderer.c:343
#3 0x55b9aedb75ee in main /sysprog/spg_part2_main.cpp:43
#4 0x7f9ad6f32b96 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x21b96)
```

Deze stack trace toont aan welke code het geheugen aangemaakt heeft. Zoals je ziet worden alle geheugenoperaties uiteindelijk uitgevoerd door `libasan`, je mag regel #0 dus over slaan. Het eigenlijke geheugenlek doet zich voor in de `graphics_init` functie in `opengl_game_renderer.c` op lijn 364. De rest van de stacktrace toont de volledige volgorde van oproepen: de `main` functie roept op lijn 43 de `graphics_alloc` functie op, die op lijn 343 de `graphics_init` functie oproept.

Bij het debuggen wordt een extern consolevenster gebruikt dat afsluit bij het stoppen van het programma. Hierdoor is het niet mogelijk om de memory leaks te zien na een debugsessie. Start een normale uitvoering met het programma door “Terminal > Run Build Task” uit te voeren (`ctrl-shift-b`) en “run sys_plex_part1 (+ build)” uit te voeren. Dit zal het project eerst builden en dan uitvoeren. De output zie je in het terminalvenster van Visual Studio Code.

Merk op dat de optie “start without debugging” de debugger toch opstart. Dit is een gekende fout in de c/c++ extensie van Visual Studio Code.

BIJLAGE F PLATFORM-ONAFHANKELIJK PROGRAMMEREN

Het project moet zowel op de Raspberry Pi als op andere Linux machines uitvoerbaar zijn.

Hiervoor moet rekening gehouden worden met de verschillen tussen deze platformen. Deze appendix geeft een overzicht van enkele manieren om dit te doen.

F.1 BUILD SYSTEMS

Build systems zijn verantwoordelijk voor het oproepen van de compiler en linker met de correcte argumenten. Als je een IDE gebruikt is het build system er dikwijls een onderdeel van. Gewone editors zoals Visual Studio Code gebruiken echter externe build tools zoals de `make` build tool. Elk build system houdt in 1 of meerdere files bij hoe het project in elkaar zit. Van deze info wordt gebruikt gemaakt bij het compileren. Een build system zou platformonafhankelijk kunnen zijn, maar in

praktijk zit er dikwijls nog een laag boven, die de project files voor het build system genereert. Dat is bijvoorbeeld typisch het geval met autoconf en automake op Linux (waar je dan als eerste stap `./configure` uitvoert om de makefiles te genereren).

Voor dit project maken we gebruik van “CMake” om de build files te genereren. CMake genereert platform-afhankelijke MakeFiles die op hun beurt dan gebruikt worden om het eigenlijke project te bouwen. De verschillen tussen platformen worden dus grotendeels door CMake opgevangen.

De file `CmakeLists.txt` bevat alle project info die CMake nodig heeft. Het is mogelijk om in die file bepaalde dingen slechts op bepaalde platformen uit te voeren. Op details omtrent CMake gaan we hier niet verder in.

F.2 PREPROCESSOR MACRO'S

In de C en C++ code van het project zijn sommige stukken code afhankelijk van het platform. Om dit op te vangen wordt typisch gebruik gemaakt van preprocessor macro variabelen die enkel op specifieke platformen beschikbaar zijn. De macro variabele `WIN32` is enkel op Windows systemen aanwezig (ook indien mingw en dergelijke gebruikt worden). De macro variabele `LINUX` is enkel op Linux systemen aanwezig. De CMake files zijn zo aangepast, dat ze de macro variabele `RPI` toevoegen, die dus voor het project enkel op Raspberry Pi aanwezig zal zijn. We kunnen platformspecifieke code toevoegen door de `#ifdef` constructie van de C preprocessor te gebruiken:

```
#ifdef RPI
    engine->orientation_system = init_orientation_system();
#endif
#ifdef _WIN32
    _CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);
#endif
```

F.3 PLATFORMONAFHANKELIJKE INTERFACES

Een manier om code te schrijven die overal hetzelfde is, is om gebruik te maken van platformonafhankelijke interfaces. Deze zijn door een standaardisatie groep gedefinieerd, en worden op de verschillende platformen geïmplementeerd. Een voorbeeld is POSIX, dat o.a. specificeert hoe in C threads aangemaakt kunnen worden. Een relevant voorbeeld voor het project is OpenGL. Deze interface wordt ondersteund door de makers van grafische chipsets, waardoor verschillende GPU's via dezelfde software kunnen worden gebruikt.

F.4 PLATFORMONAFHANKELIJKE BIBLIOTHEKEN

Een aantal bibliotheken is beschikbaar op verschillende platformen, met als doel op elk platform dezelfde code te kunnen gebruiken. De verschillen tussen de platformen worden opgevangen in de implementatie van de bibliotheek. Een voorbeeld in het project is de SDL bibliotheek, die input van muis, toetsenbord en joystick, en output naar scherm en audio aanbiedt op verschillende platformen. Ook voor GUI's zijn platformonafhankelijke bibliotheken beschikbaar, zoals Qt en GTK+.

F.5 PLATFORMONAFHANKELIJKHEID VIA C STANDARD FEATURES

De C en C++ standaard bibliotheek bevat verschillende definities en functies om makkelijker platformonafhankelijk te programmeren. De lengte van het `int` datatype is in C afhankelijk van het platform. De `stdint.h` header die in de C standaard gedefinieerd is, heeft verschillende types die wel een platformonafhankelijke lengte hebben, zoals bv `int32_t`. C++17 biedt functionaliteit voor het werken met threads aan, waardoor er geen platform afhankelijke bibliotheken vereist zijn.

F.6 ENDIAN AWARENESS

Om volledig platformonafhankelijk te zijn, is het nodig om endian-aware code te schrijven bij het schrijven en lezen van bestanden en bij netwerkcommunicatie. Het omzetten van een `unsigned short` in “host system” order (typisch little

endian op x86 hardware) naar big endian, kan met de functie `htons()`. Omzetten van big endian naar "host system" endian kan met `ntohs()`. (Merk op dat de "n" hier voor "network" staat: big endian is de volgorde voor netwerk communicatie.) Als hier geen aandacht aan besteed wordt, zal op big-endian systemen een bestand weggeschreven worden dat niet compatibel is met little-endian systemen, en vice versa.

Rekenoperaties en bitshifts gebeuren steeds correct volgens de endianness van het systeem. Hier kan gebruik van gemaakt worden, zo zal volgende code steeds de meest en minst significante byte van een 32-bit getal in een 8-bit variabele plaatsen:

```
#include <stdint.h>
#include <stdio.h>

// include htonl, ntohl, ntohs en htons
#include <netinet/in.h>

int main() {
    //input wordt verschillend opgeslagen in geheugen afh van endianness!
    uint32_t input = 0x01020304;

    //endian independant code
    uint8_t least_sig_byte = input & 0xFF; //steeds 0x04
    uint8_t most_sig_byte = (input >> 24) & 0xFF; //steeds 0x01

    //endian independant code
    uint32_t big_endian_input = htonl(input);
    uint8_t least_sig_byte2 = ((uint8_t*)&big_endian_input)[3]; //steeds 0x04
    uint8_t most_sig_byte2 = ((uint8_t*)&big_endian_input)[0]; //steeds 0x01

    //endian dependant code
    uint8_t last_byte = ((uint8_t*)&input)[3];
    uint8_t first_byte = ((uint8_t*)&input)[0];
    //last_byte is 0x01 on little endian, 0x04 on big endian
    //first_byte is 0x04 on little endian, 0x01 on big endian

    printf("Input: 0x%X\n", input);
    printf("Most: %X Least: %X\n", most_sig_byte, least_sig_byte);
    printf("Most2: %X Least2: %X\n", most_sig_byte2, least_sig_byte2);
    printf("First: %X Last: %X\n", first_byte, last_byte);
}
```

Output op een little endian systeem:

```
Input: 0x1020304
Most:  1  Least:  4
Most2: 1  Least2: 4
First: 4  Last:   1
```

Output op een big endian systeem:

```
Input: 0x1020304
Most:  1  Least:  4
Most2: 1  Least2: 4
First: 1  Last:   4
```

Ten slotte nog een overzicht van de endianness van enkele systemen:

- Intel x86 en x86-64 CPUs (of compatibele): little endian
- De meeste ARM CPU's: configureerbaar (= bi-endian), maar typisch little endian by default
- Oudere Power PCs (OS X): big endian
- Xbox 360 & PS 3: big endian
- Xbox One & PS 4: little endian
- MIPS (legacy processor architecture): big endian
- SPARC (legacy processor architecture van Sun Microsystems): big endian
- Raspberry Pi: little endian
- iPhone en Android smartphones: little endian

BIJLAGE G COMPILEREN

Wij raden aan om dit project te ontwikkelen met Visual Studio Code en de GCC 10 compiler. Deze tools worden ook voor het examen gebruikt waardoor het nuttig is om vertrouwd te raken met de warnings en foutmeldingen van deze compiler en de gebruikersinterface en debugger van Visual Studio Code.

De Systeemprogrammeren documentatie bevat een uitgebreide uitleg over hoe je de ontwikkelingsomgeving en bijhorende tools installeert: <https://github.ugent.be/Systeemprogrammeren/docs/blob/master/README.md>. Daarnaast legt deze documentatie ook uit hoe je het project compileert en debugged. Volg deze instructies zo exact mogelijk!

BIJLAGE H CALLING CONVENTIONS

Dit is achtergrondinformatie, die niet essentieel is voor het project, maar wel interessant.

De meegegeven assembler programmas maken gebruik van functie calls, maar doet dit niet op de klassieke manier dat C code gegenereerd vanuit een C compiler dit zou doen.

Typisch wordt bij x86 architectuur bij het compileren van C en C++ code, de "cdecl calling convention" gebruikt. Een calling convention is een afspraak die er voor zorgt dat binaire code afkomstig van verschillende compilers elkaars C functies kunnen oproepen. De afspraak bepaalt hoe code een functie kan oproepen, hoe parameters en resultaten worden doorgegeven, wie welke registers mag aanpassen etc.

De "cdecl calling convention" maakt volgende afspraken:

- Functie argumenten worden in omgekeerde volgorde op de stack geplaatst (en worden dus niet via registers doorgegeven).
- Via het `%eax` register wordt het resultaat van de functie (integer waarde of address) teruggegeven. (In geval van floating point wordt een ander register gebruikt)
- 3 specifieke registers zijn vrij te gebruiken door de oproepende functie. Dat betekent dat de code die de functie oproept deze registers voor de functie moet opslaan in het geheugen, en achteraf moet herstellen, tenzij de registers niet gebruikt worden. (= "caller saved registers")
- De andere registers mogen niet door de functie aangepast worden (voor zover de oproepende code kan zien). Dat betekent dat de functie de registers wel kan gebruiken, maar moet zorgen dat de oorspronkelijke waarde hersteld wordt voor de functie retourneert. (= "callee saved registers")
- De code die de functie oproept moet als de functie voltooid is, de argumenten van de stack verwijderen.

Dit is uiteraard maar een beknopt overzicht van cdecl. In praktijk worden `%sp` en `%bp` ook nog betrokken, en zijn er nog enkele niet gedefinieerde gevallen waardoor cdecl niet steeds bij elke compiler hetzelfde werkt.

Er bestaan ook vele andere calling conventies, afhankelijk van CPU architectuur, programmeertaal en operating system. Zo is er een specifieke calling convention voor ARM (dus o.a. raspberry Pi) op Linux.

BIJLAGE I WERKVERDELING

Het werk aan deel 1 van het project kan in verschillende stukken opgesplitst worden. Deze kunnen verdeeld worden tussen de groepsleden. Uiteraard varieert de hoeveelheid werk tussen de stukken.

Verdeel het werk op voorhand, maar wees vooral flexibel als gaandeweg blijkt dat de ene meer werk kreeg dan de andere.

Het kan ook zijn dat je moet wachten op het werk van je partner, overleg dan, en kijk of er iets anders is dat <https://www.overleaf.com/project/62f1f514> wel kan doen, en of je eventueel kan helpen.

Een voorbeeld van in welke deelstukken het project gesplitst kan worden:

Main (1 stuk): Argumenten verwerken en doorgeven aan entity system.

Inlezen levels (1 stuk): Lezen van binaire level files en omzetten naar Level struct.

Assemblage (vele stukken): De `struct Level` wordt door de assemblage naar de verschillende entities en components omgezet. De basisomzetting gebeurt in de gegeven functie `set_level_entities` (specifieker in `create_tile_entities`), die verschillende functies om entities aan te maken oproept. Deze functies kunnen allemaal verdeeld worden. Degene die een system schrijft, zal typisch deze functies aanpassen en vervolledigen waar nodig.

Systems (vele stukken): Systems hebben veelal aparte functionaliteit, en kunnen dus onderling verdeeld worden. Je kan tijdens het implementeren merken dat sommige systemen wat samenhangen. Dan is het aan te raden dat dezelfde persoon ze implementeert, of dat er zeer nauw samengewerkt wordt. Sommige systemen zijn ook meer werk dan andere.

Level editor (1 stuk): De editor kan volledig afzonderlijk van de rest van deel 1 geschreven worden. We raden echter aan om niet te starten met dit deel.

Ook deel 2 kan in verschillende onafhankelijke stukken gesplitst worden:

Implementeren bitwriter en bitreader (2 stukken): Implementeren van de bitwriter en bitreader. Merk op dat dit bitreader noodzakelijk is om de CPU instructies te kunnen testen. Deze moet dus vroeg geïmplementeerd worden.

Implementeren CPU instructies (meerdere stukken): Implementeren van de CPU instruction. De instructies kunnen onderling verdeeld worden. Merk wel op dat er gelijkaardige instructies zijn, die dus makkelijk samen geïmplementeerd worden.

Systems voor carbrain. (2 stukken): Implementeren van het carbrain sensor en carbrain decision system, die deel 1 met deel 2 linken.

Testen (0 of 1 stuk): Het is handig om tijdens het implementeren van de CPU instructies de bijhorende testen te schrijven. Het kan echter ook interessant zijn als iemand op voorhand alle testen schrijft, en iemand anders dan implementeert en daarbij gebruik maakt van de testen.

BIJLAGE J TIPS OM VLOT TE STARTEN

Het is altijd best om te beginnen met een zo klein mogelijke functionaliteit, deze te testen, en vervolgens uit te breiden.

Een voorbeeld hoe dit zou kunnen voor de projectopgave:

Implementeren van een system:

Stap 1: Denk na over wat het system moet doen. Welke components verwacht het, en hoe manipuleert het ze?

Stap 2: Vul de component structs in

Stap 3: Pas indien nodig de assemblage aan zodat de juiste components worden aangemaakt.

Stap 4: Implementeer een eerste deel van het system. Begin met de kleinste functionaliteit die je kan testen.

Stap 5 en verder: Test het system, verbeter het, en voeg functionaliteit toe.